

DEPARTAMENTO
DE COMPUTACION

Facultad de Ciencias Exactas y Naturales - UBA

Fundamentos de arquitectura de computadoras

ISA, ciclo de instrucción y RISC-V

Sistemas Digitales

2do Cuatrimestre 2026

FCEN - UBA

Introducción

Sobre la clase de hoy

Hoy vamos a ver los fundamentos de la computadora, su especificación formal y una arquitectura concreta

Sobre la clase de hoy

Hoy vamos a ver los fundamentos de la computadora, su especificación formal y una arquitectura concreta

- **La arquitectura de Von Neumann**

Sobre la clase de hoy

Hoy vamos a ver los fundamentos de la computadora, su especificación formal y una arquitectura concreta

- **La arquitectura de Von Neumann**
- **Instruction Set Architecture**

Sobre la clase de hoy

Hoy vamos a ver los fundamentos de la computadora, su especificación formal y una arquitectura concreta

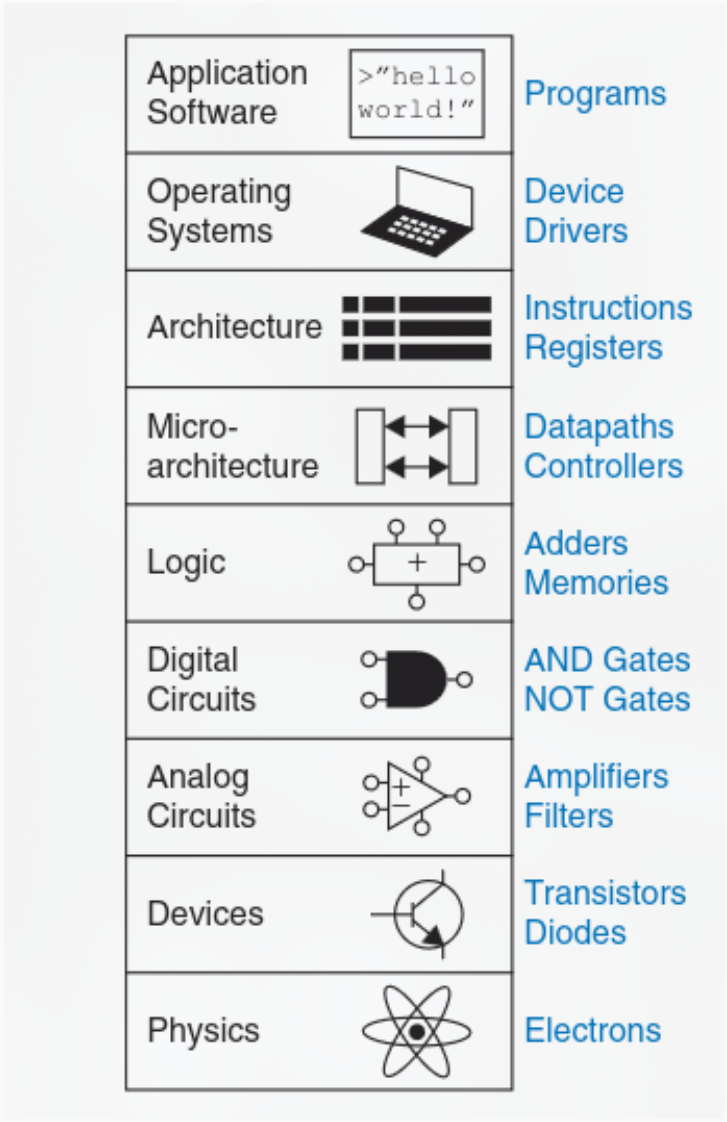
- **La arquitectura de Von Neumann**
- **Instruction Set Architecture**
- **Ciclo de instrucción**

Sobre la clase de hoy

Hoy vamos a ver los fundamentos de la computadora, su especificación formal y una arquitectura concreta

- **La arquitectura de Von Neumann**
- **Instruction Set Architecture**
- **Ciclo de instrucción**
- **El caso RISC-V**

¿Dónde estamos parados?



- **Ya vimos:** *Diseño digital* y *Lógica*
- **Hoy:** *Arquitectura* (la ISA)
- **Próxima:** *Microarquitectura*

Harris & Harris, Digital Design and Computer Architecture: RISC-V Edition, *2021*

Arquitectura de Von Neumann

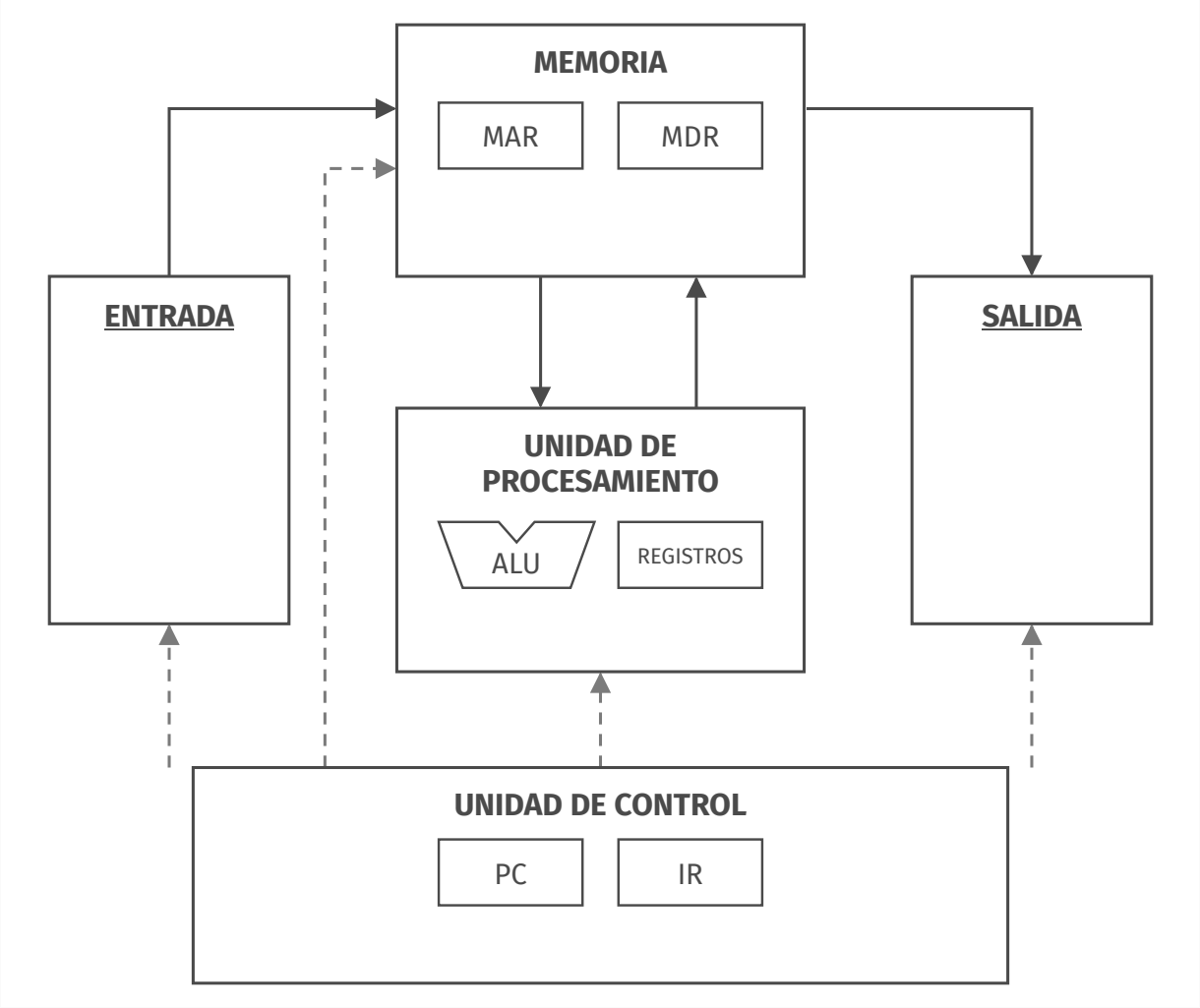
John von Neumann



John von Neumann

Matemático de origen húngaro y nacionalidad estadounidense (1903-1957). Pionero en múltiples ámbitos de la ciencia como la lógica, la mecánica cuántica y la teoría de juegos, entre otros. En 1945 participó en el proyecto EDVAC y escribió *First Draft of a Report on the EDVAC*, en donde propone un modelo de ejecución fundamental para programas de computadora.

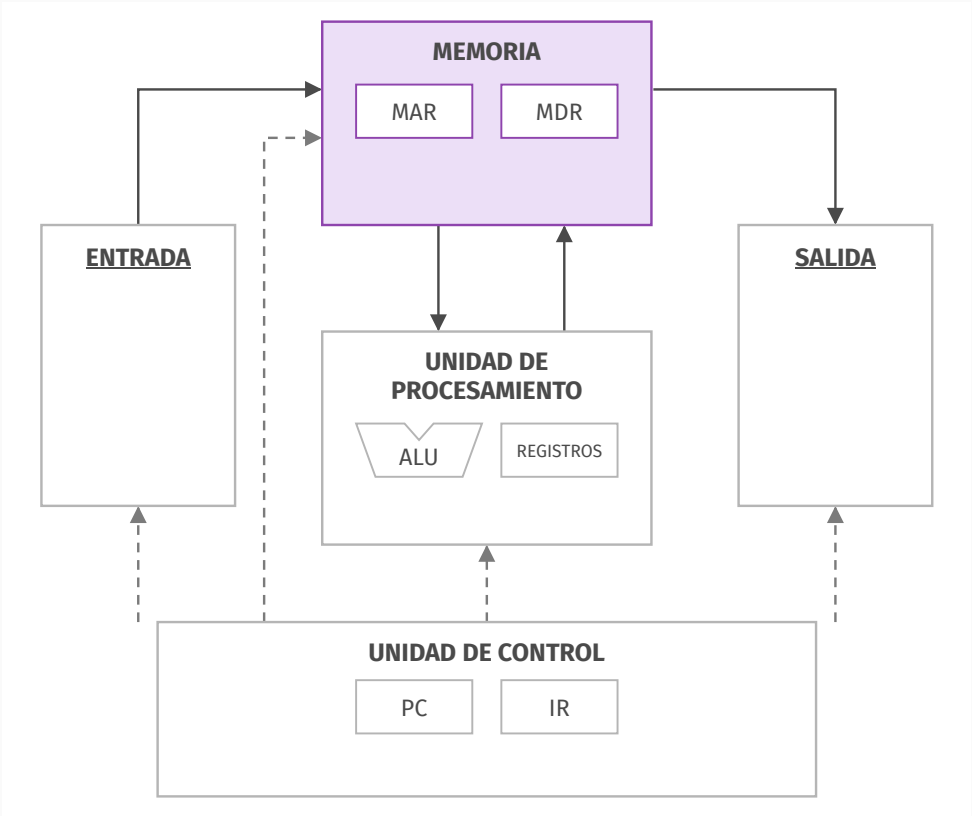
Estructura de una computadora



Memoria, unidad de procesamiento, entrada, salida y unidad de control

Adaptado de Patt & Patel, Introduction to Computing Systems, *3ra ed., 2020, capítulo 4*

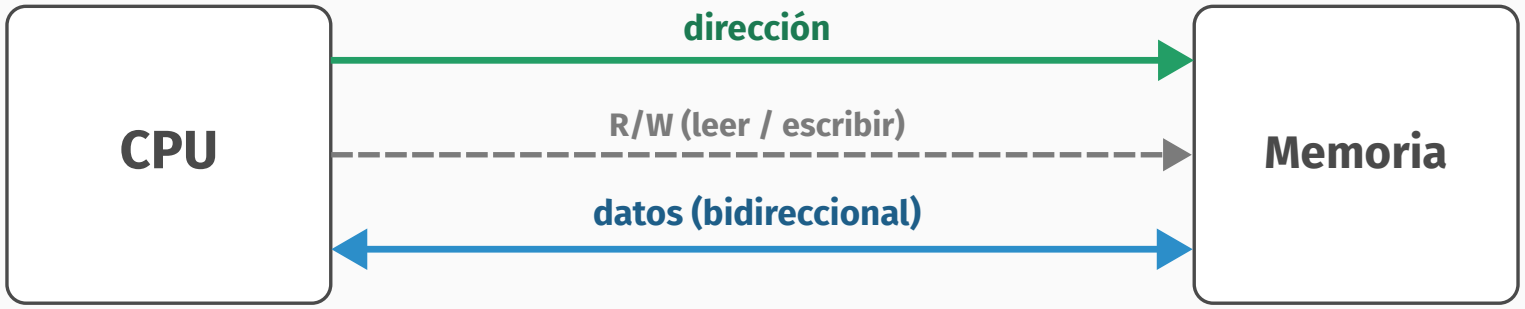
Estructura de una computadora: memoria



Es un almacenamiento en donde tanto **programas** como **datos** van a ser guardados en el mismo lugar. El programa es una secuencia de **instrucciones** que la CPU lleva a cabo para realizar cierta tarea.

Estructura de una computadora: memoria

Para la CPU, la memoria es un componente externo con el que interactúa mediante direcciones de memoria y datos para leer o escribir.



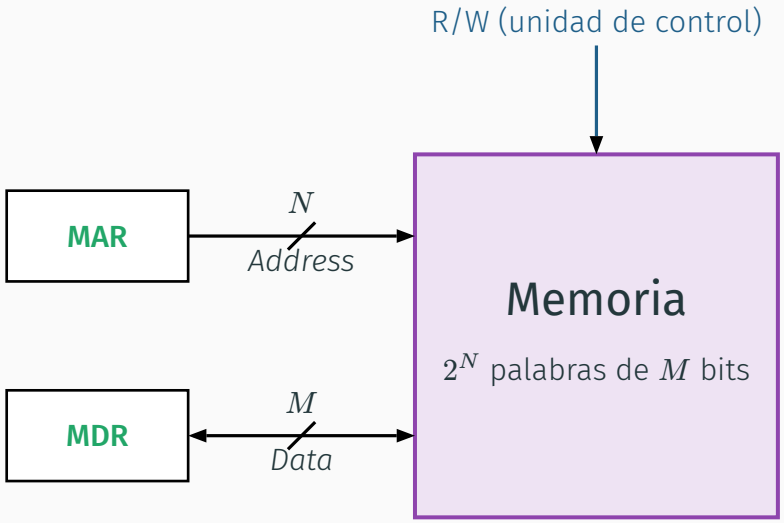
Lectura (R/W = 1)

1. CPU pone la dirección en el bus
2. Memoria responde con el dato en el bus de datos

Escritura (R/W = 0)

1. CPU pone dirección y dato
2. Memoria guarda el dato en esa dirección

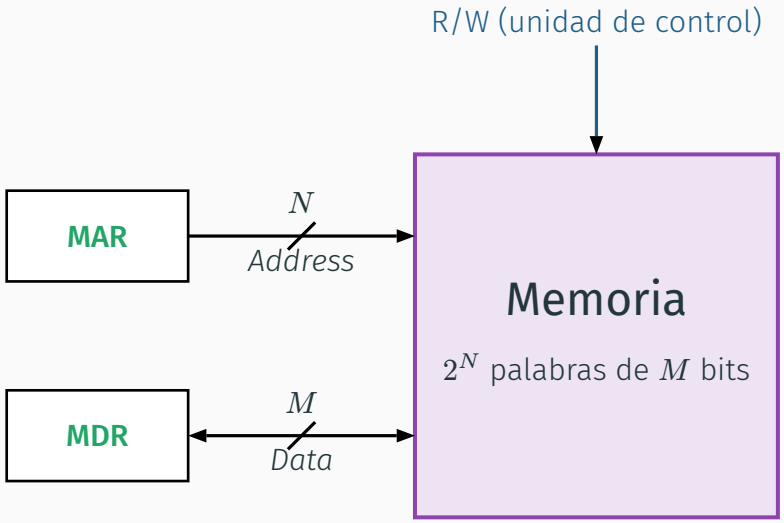
Estructura de una computadora: memoria



La memoria de la computadora es un array como el de la clase pasada:

- La CPU pone la dirección en el **MAR** (N bits)
- El dato pasa por el **MDR** (M bits)
- La unidad de control indica si se lee o se escribe

Estructura de una computadora: memoria

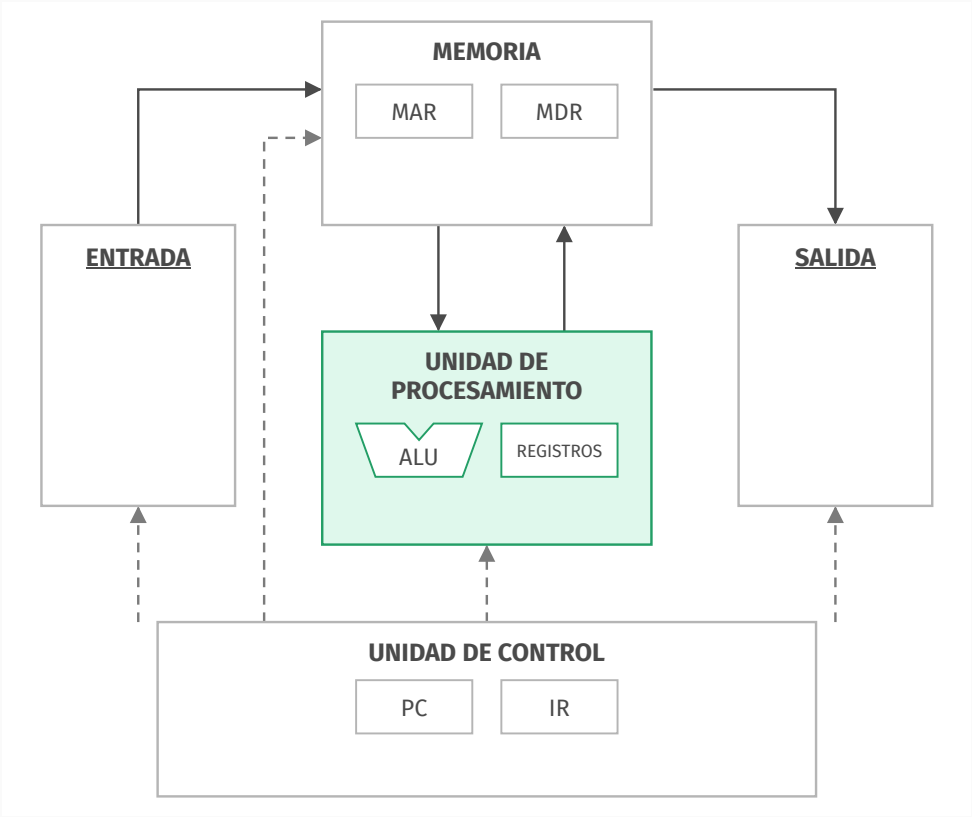


La memoria de la computadora es un array como el de la clase pasada:

- La CPU pone la dirección en el **MAR** (N bits)
- El dato pasa por el **MDR** (M bits)
- La unidad de control indica si se lee o se escribe

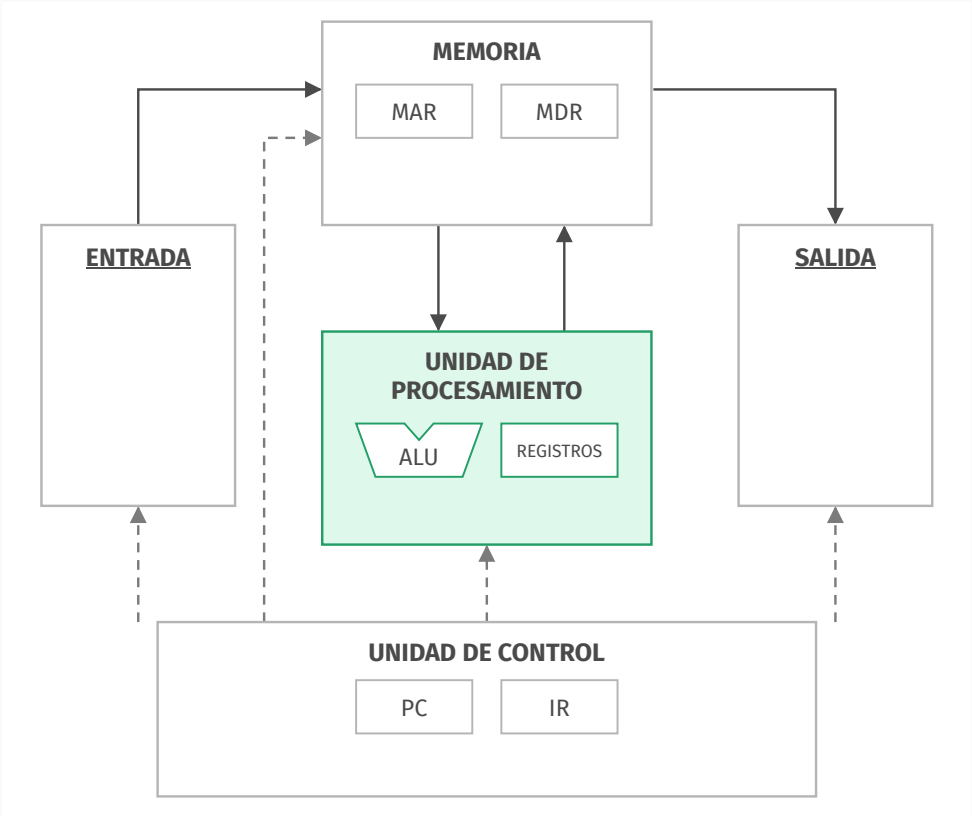
- Con direcciones de 32 bits y celdas de 1 byte se direccionan 2^{32} bytes = 4 GiB
- Un dato de 32 bits ocupa 4 celdas seguidas

Estructura de una computadora: procesamiento



Es la parte del procesador que ejecuta las operaciones sobre los datos

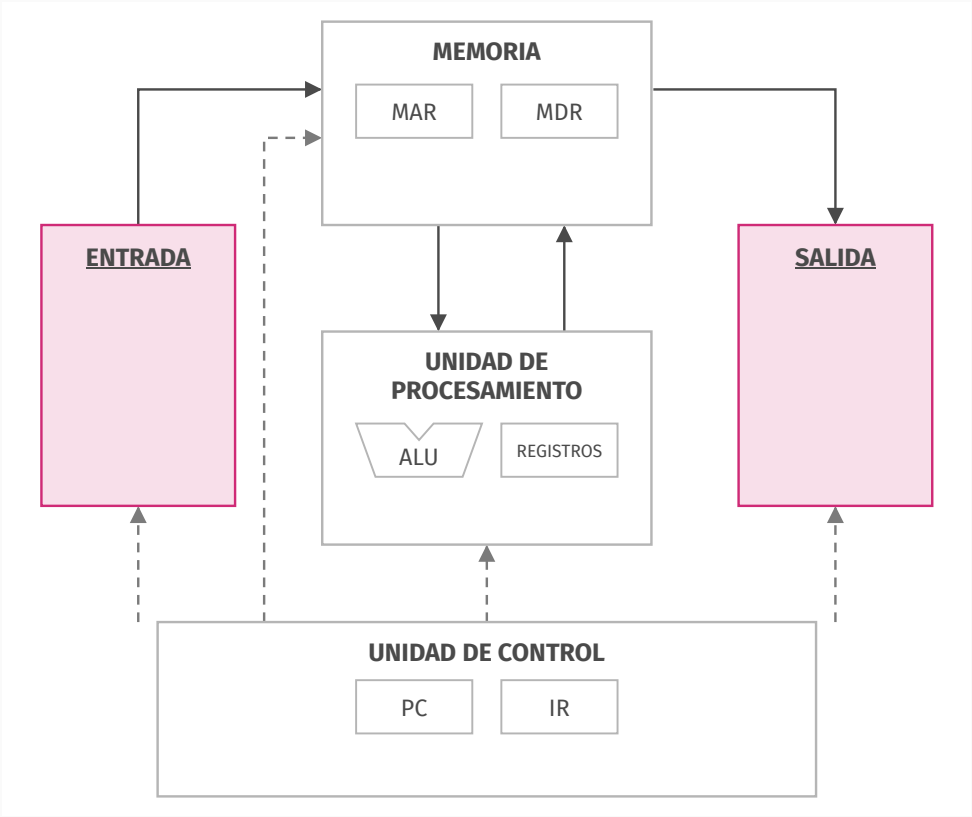
Estructura de una computadora: procesamiento



Es la parte del procesador que ejecuta las operaciones sobre los datos

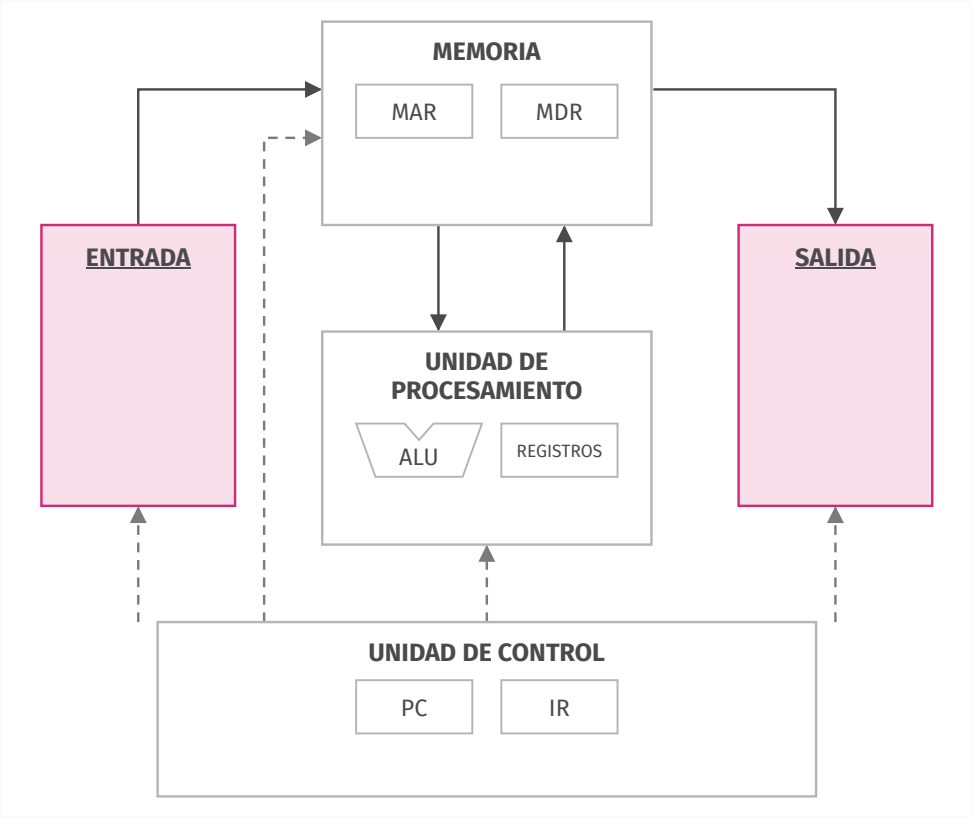
- Una o más **ALUs**, según el tipo de dato: enteros, punto flotante, vectores
- Un banco de **registros**: de tamaño fijo y de acceso mucho más rápido que la memoria

Estructura de una computadora: entrada y salida



La entrada y salida definen un mecanismo por el cual se pueden ingresar datos al sistema, procesarlos, y luego extraerlos.

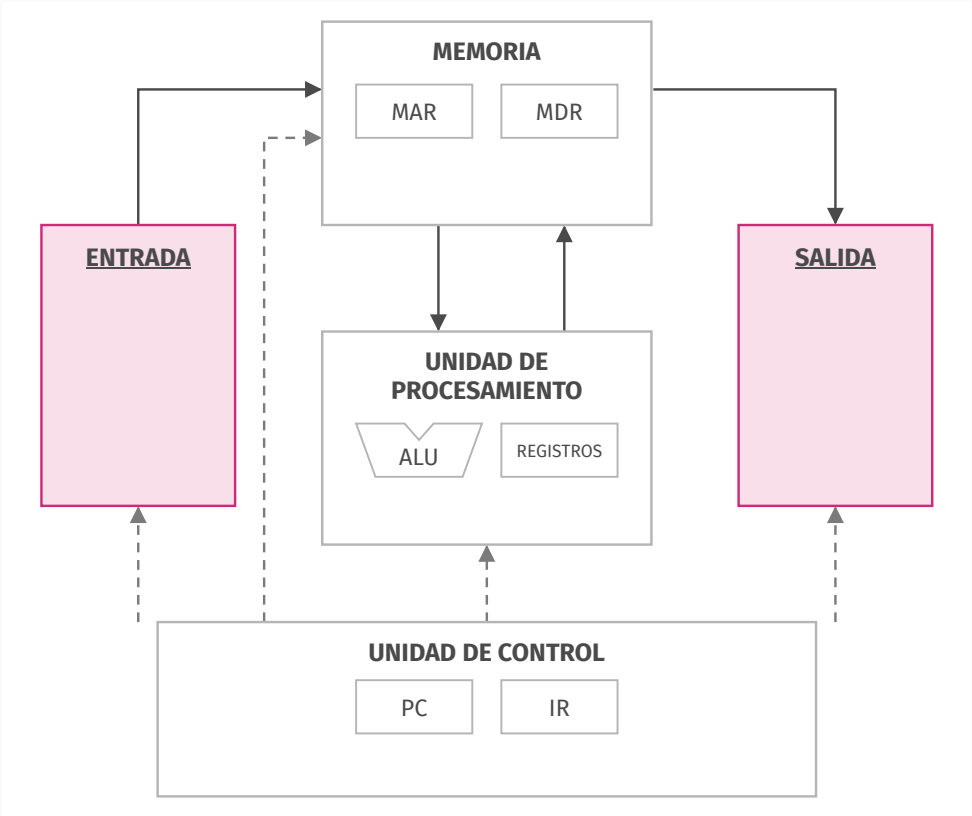
Estructura de una computadora: entrada y salida



La entrada y salida definen un mecanismo por el cual se pueden ingresar datos al sistema, procesarlos, y luego extraerlos.

Algunos componentes de entrada-salida:

Estructura de una computadora: entrada y salida

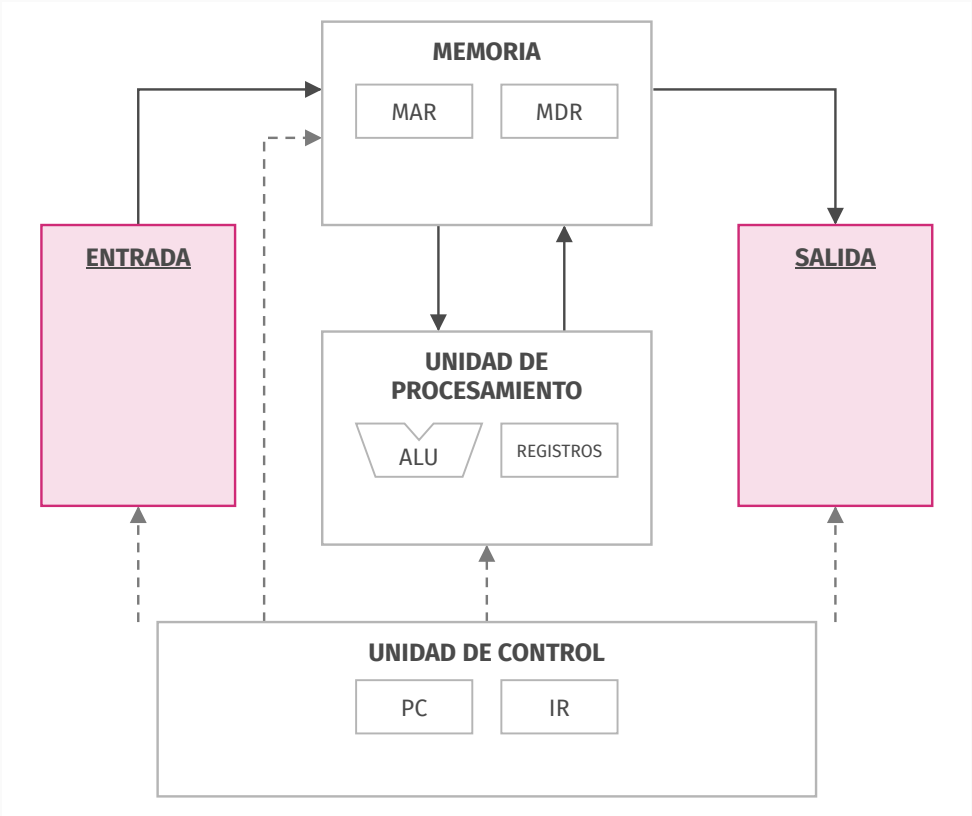


La entrada y salida definen un mecanismo por el cual se pueden ingresar datos al sistema, procesarlos, y luego extraerlos.

Algunos componentes de entrada-salida:

- Mouse/Teclado

Estructura de una computadora: entrada y salida

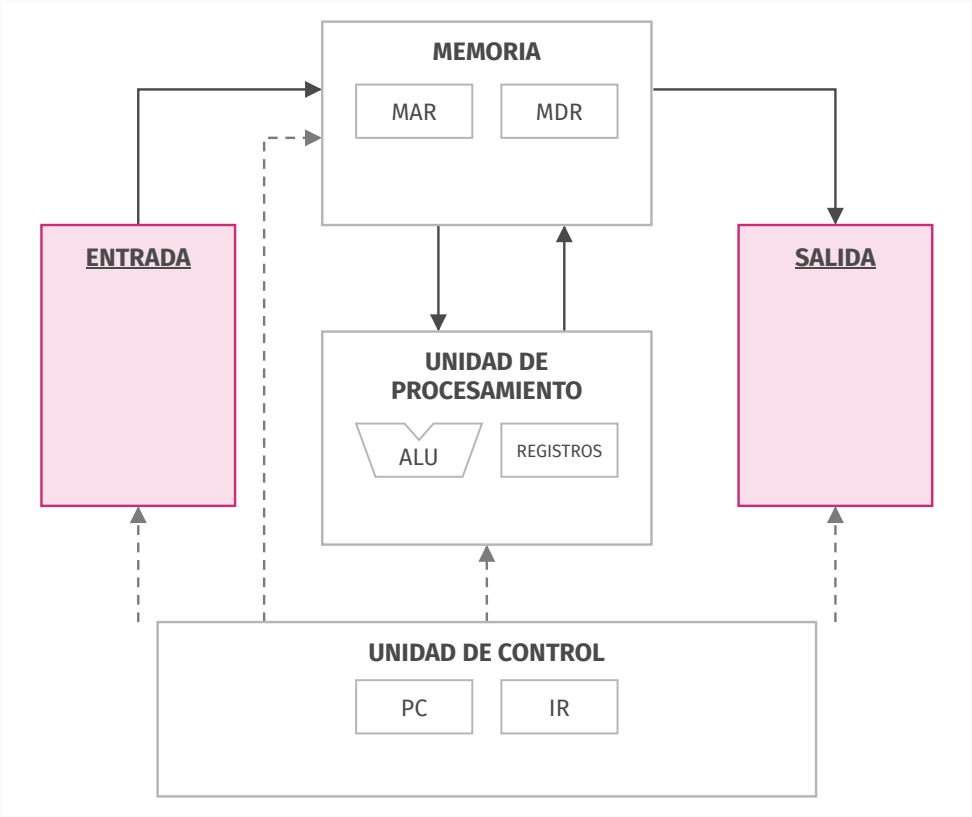


La entrada y salida definen un mecanismo por el cual se pueden ingresar datos al sistema, procesarlos, y luego extraerlos.

Algunos componentes de entrada-salida:

- Mouse/Teclado
- GPUs

Estructura de una computadora: entrada y salida

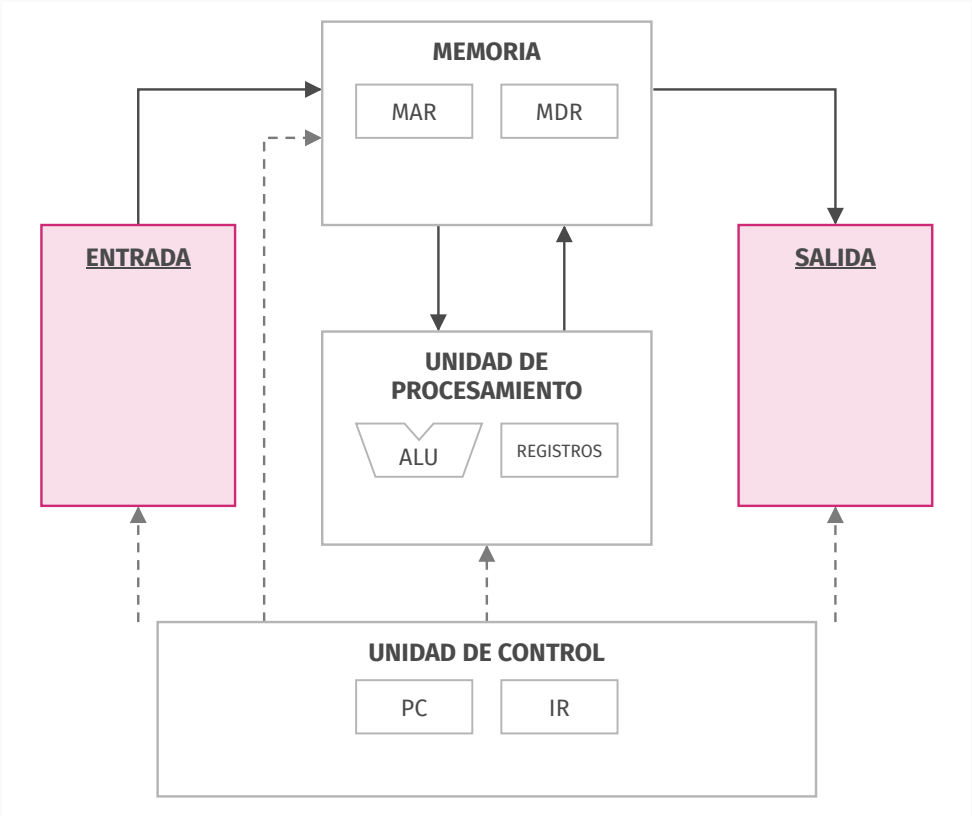


La entrada y salida definen un mecanismo por el cual se pueden ingresar datos al sistema, procesarlos, y luego extraerlos.

Algunos componentes de entrada-salida:

- Mouse/Teclado
- GPUs
- Disco externo

Estructura de una computadora: entrada y salida



La entrada y salida definen un mecanismo por el cual se pueden ingresar datos al sistema, procesarlos, y luego extraerlos.

Algunos componentes de entrada-salida:

- Mouse/Teclado
- GPUs
- Disco externo
- Aceleradores (TPU, NPU, etc.)

Estructura de una computadora: entrada y salida

Cada dispositivo expone sus propios registros: de **datos**, de **estado** y de **control** (prender un LED). La CPU tiene que poder leerlos y escribirlos. Existen dos formas de interactuar con componentes externos

Estructura de una computadora: entrada y salida

Cada dispositivo expone sus propios registros: de **datos**, de **estado** y de **control** (prender un LED). La CPU tiene que poder leerlos y escribirlos. Existen dos formas de interactuar con componentes externos

Mapeada a memoria (*memory-mapped*)

- Los registros del dispositivo ocupan **direcciones** del mismo espacio que la memoria
- Se acceden con las mismas lecturas y escrituras que la memoria

Estructura de una computadora: entrada y salida

Cada dispositivo expone sus propios registros: de **datos**, de **estado** y de **control** (prender un LED). La CPU tiene que poder leerlos y escribirlos. Existen dos formas de interactuar con componentes externos

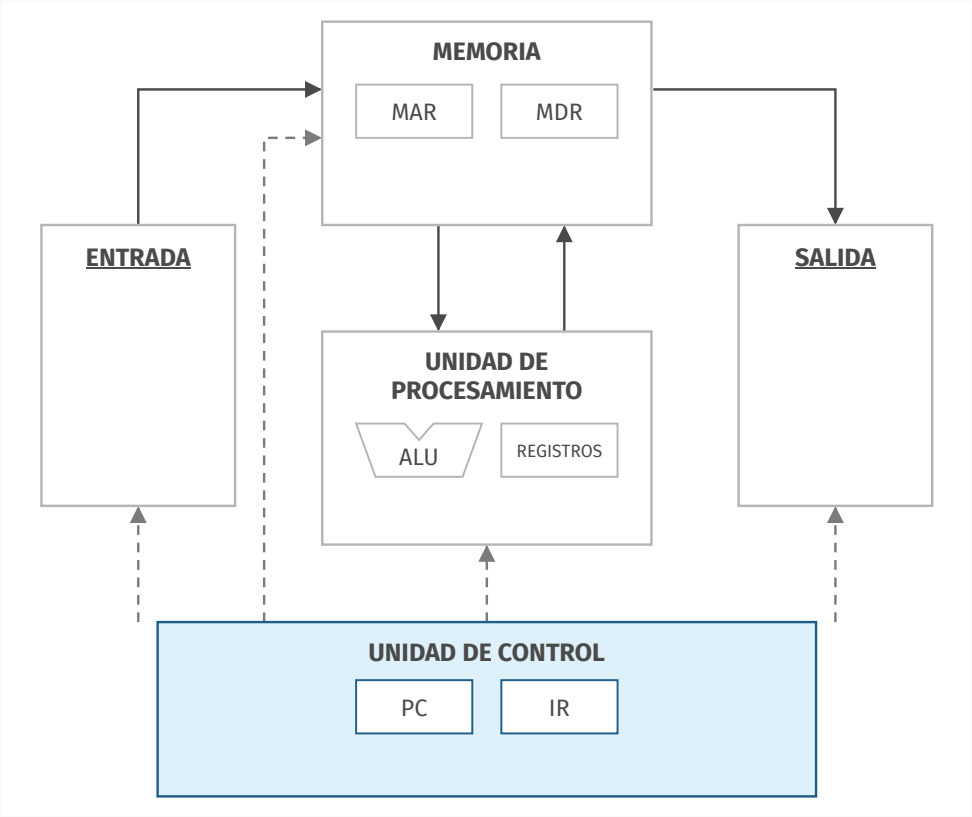
Mapeada a memoria (*memory-mapped*)

- Los registros del dispositivo ocupan **direcciones** del mismo espacio que la memoria
- Se acceden con las mismas lecturas y escrituras que la memoria

Por puertos (*port-mapped*)

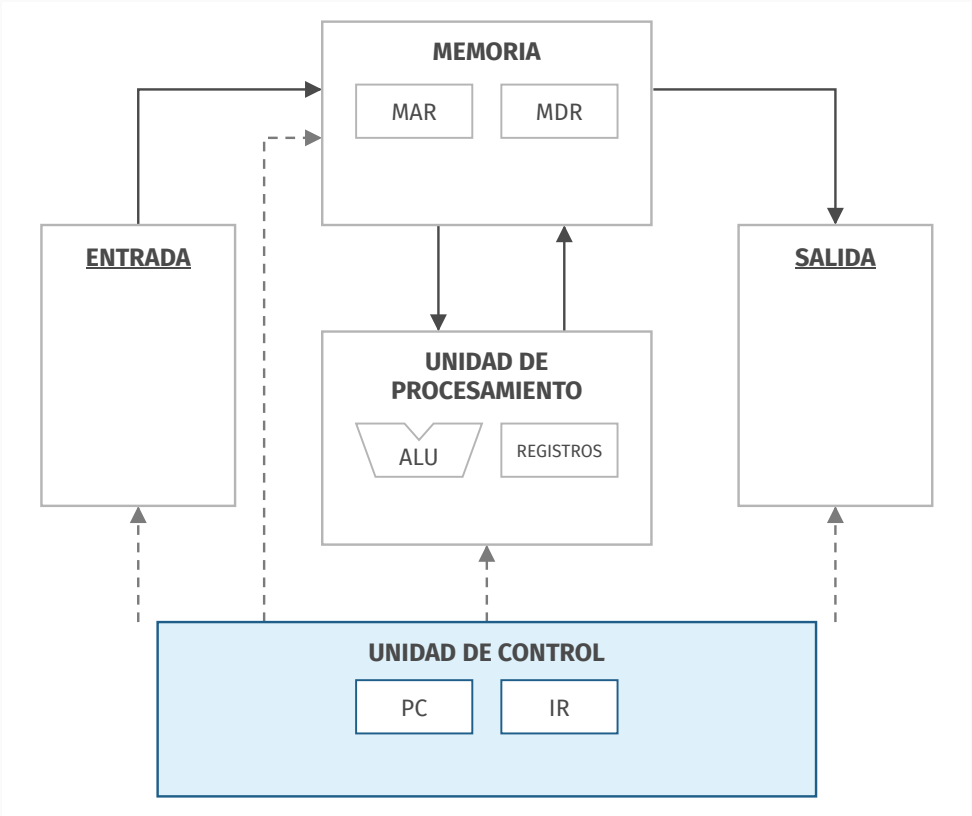
- Los dispositivos tienen un espacio de direcciones **aparte**
- Se accede utilizando instrucciones especiales del procesador

Estructura de una computadora: control



Para saber qué instrucción se está ejecutando, la unidad de control tiene un registro de instrucción que la guarda. Para saber cuál es la próxima instrucción a procesar, tiene un registro con su dirección

Estructura de una computadora: control

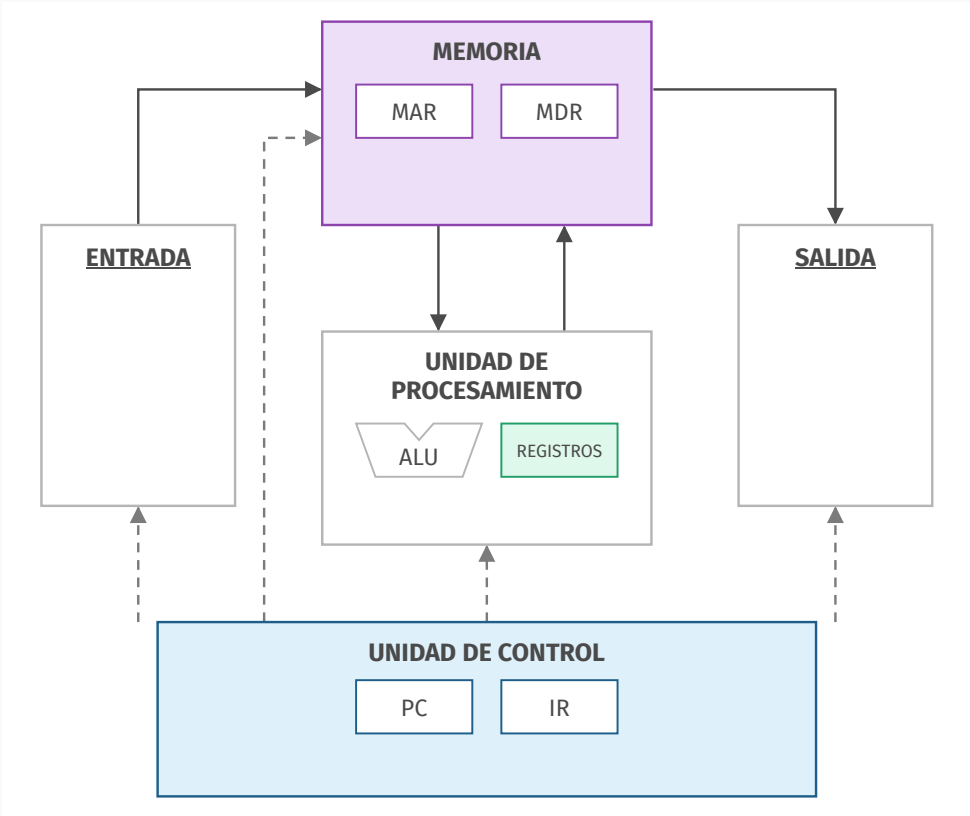


Para saber qué instrucción se está ejecutando, la unidad de control tiene un registro de instrucción que la guarda. Para saber cuál es la próxima instrucción a procesar, tiene un registro con su dirección

Dentro de la unidad de control hay dos registros importantes:

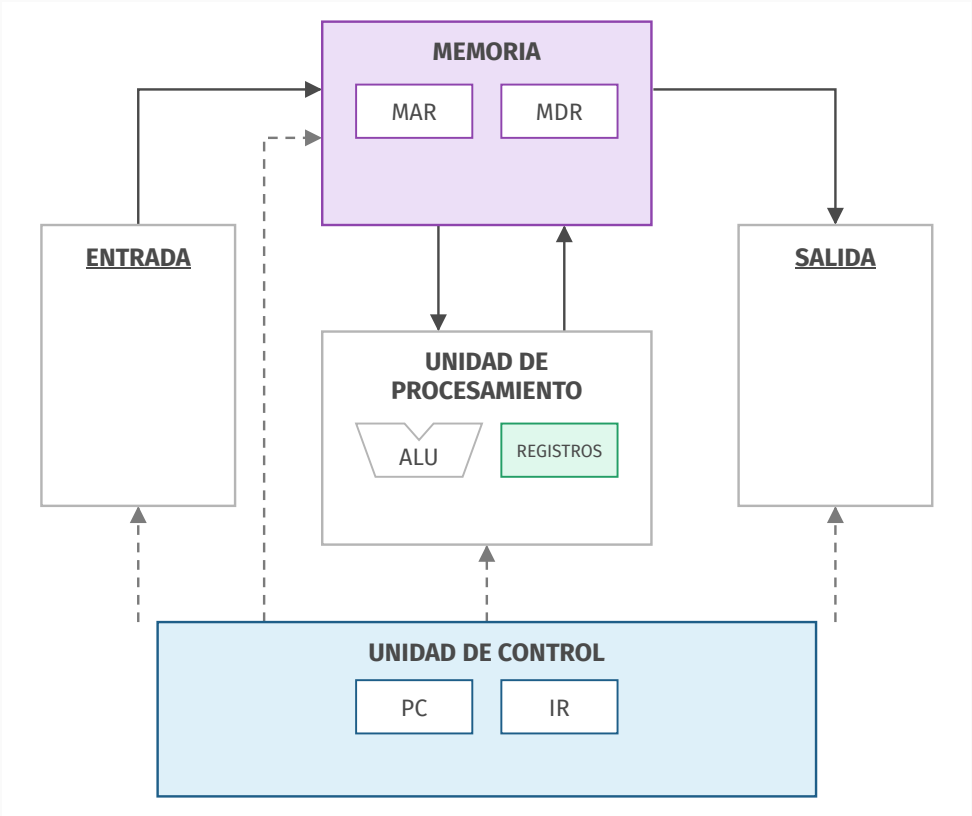
- **PC** (Program Counter): la dirección de la próxima instrucción
- **IR** (Instruction Register): la instrucción que se está ejecutando

Estado arquitectural



> ***Estado arquitectural:*** *el estado que el programador puede ver y modificar.*

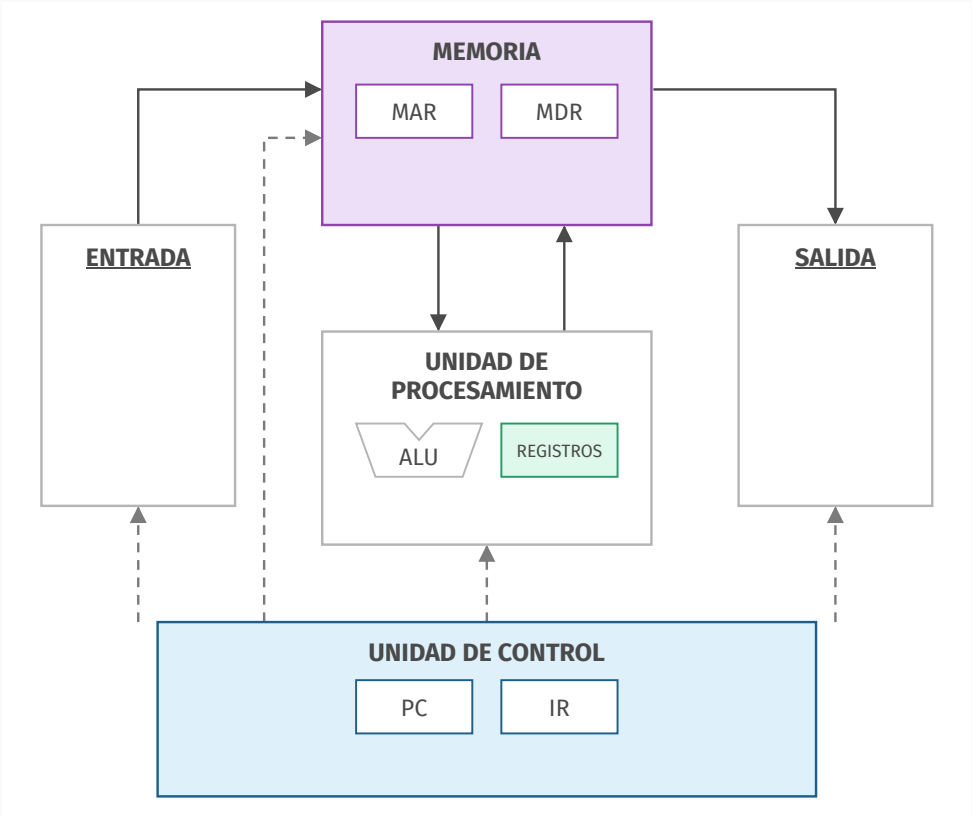
Estado arquitectural



> ***Estado arquitectural:*** *el estado que el programador puede ver y modificar.*

Dentro de este esquema, lo que tiene estado es la **memoria**, la **unidad de control** y los **registros**: son las piezas que le importan al programador

Estado arquitectural

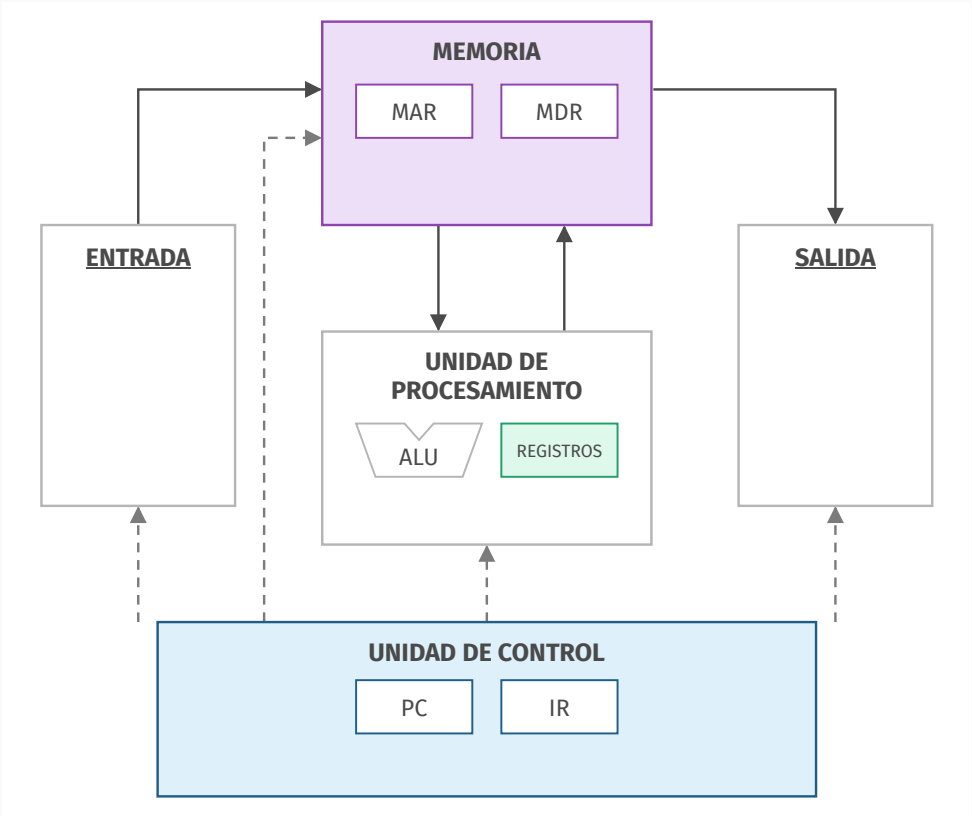


> ***Estado arquitectural:*** *el estado que el programador puede ver y modificar.*

Dentro de este esquema, lo que tiene estado es la **memoria**, la **unidad de control** y los **registros**: son las piezas que le importan al programador

- **Memoria**: datos e instrucciones
- **PC**: la próxima instrucción
- **Registros**: de propósito general y especiales (flags, control y estado)

Estado arquitectural



> ***Estado arquitectural:*** *el estado que el programador puede ver y modificar.*

Dentro de este esquema, lo que tiene estado es la **memoria**, la **unidad de control** y los **registros**: son las piezas que le importan al programador

- **Memoria**: datos e instrucciones
- **PC**: la próxima instrucción
- **Registros**: de propósito general y especiales (flags, control y estado)

Cada instrucción toma un estado arquitectural y produce el siguiente

Von Neumann: las dos ideas clave

Las dos ideas clave del modelo Von Neumann:

Von Neumann: las dos ideas clave

Las dos ideas clave del modelo Von Neumann:

Programa almacenado

- Las instrucciones y los datos se almacenan en una única memoria, como un arreglo lineal
- Un mismo valor puede ser dato o instrucción: su interpretación depende de las señales de control

Von Neumann: las dos ideas clave

Las dos ideas clave del modelo Von Neumann:

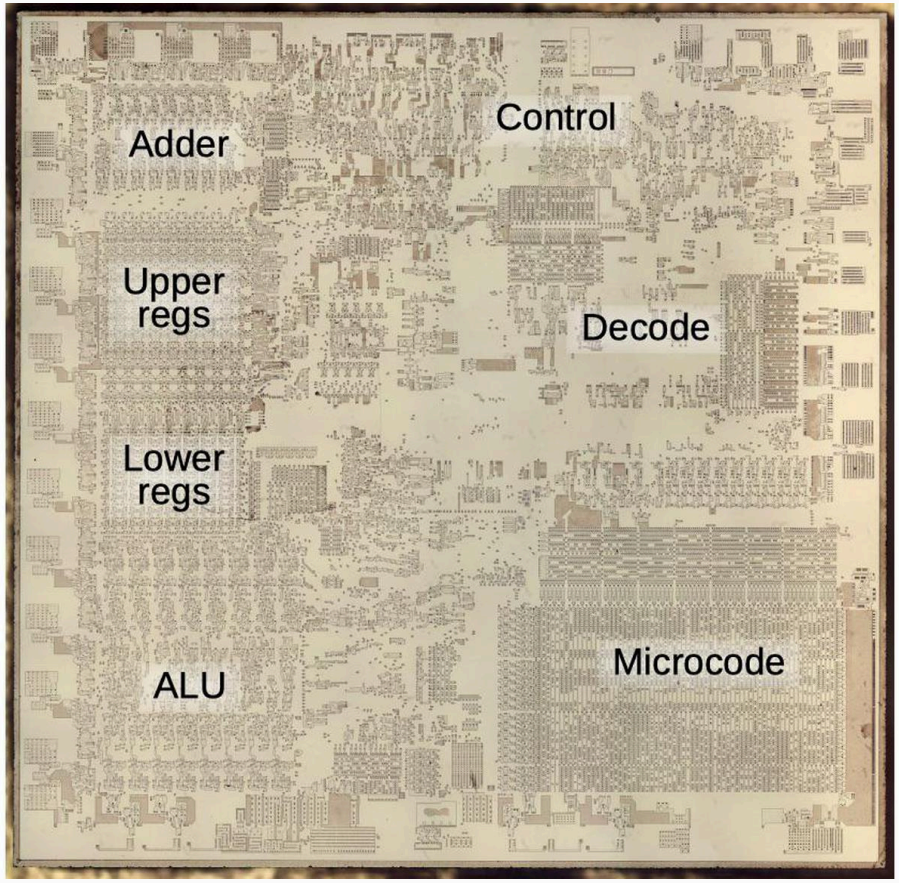
Programa almacenado

- Las instrucciones y los datos se almacenan en una única memoria, como un arreglo lineal
- Un mismo valor puede ser dato o instrucción: su interpretación depende de las señales de control

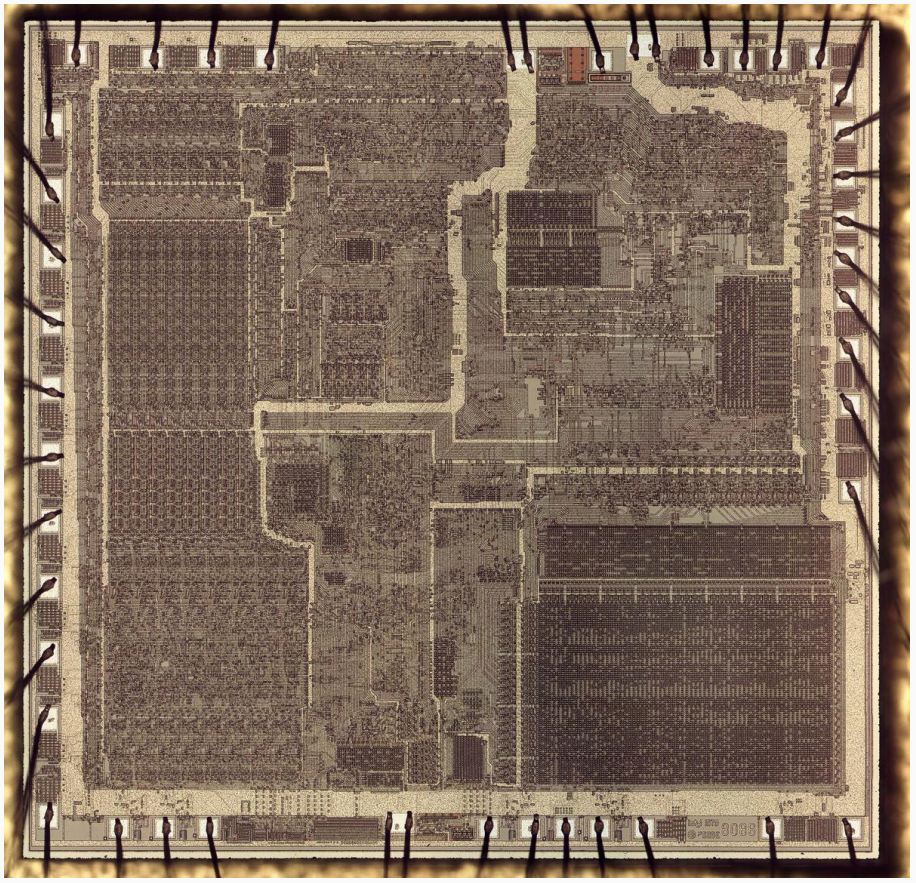
Procesamiento secuencial de instrucciones

- Se procesa una instrucción a la vez
- El **Program Counter (PC)** identifica la instrucción siguiente, y avanza secuencialmente, salvo en instrucciones de salto (transferencia de control)

Von Neumann: no es un dibujo, es silicio



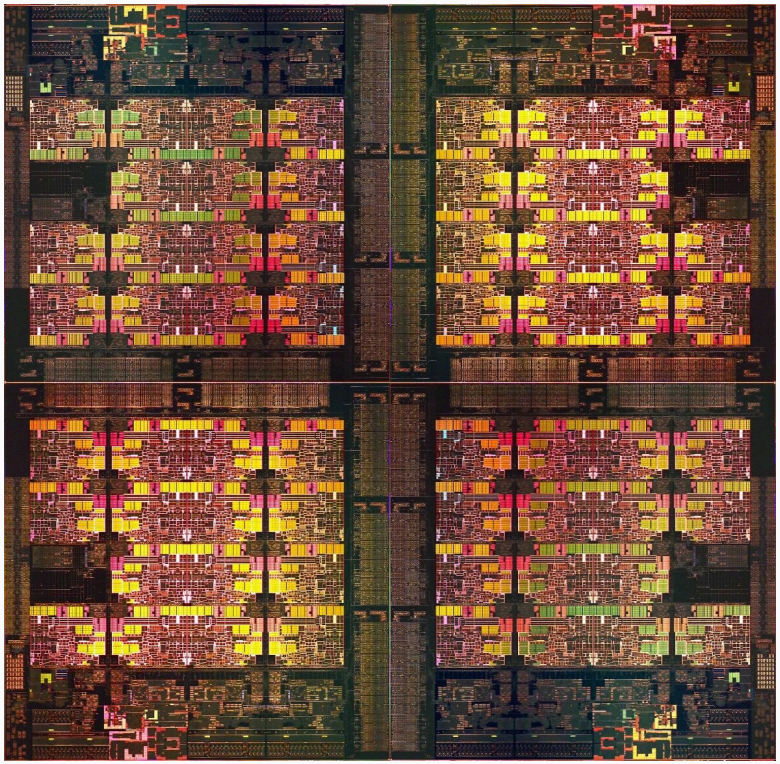
Intel 8086: las cajas del diagrama, ubicadas en el die



Intel 8086: el mismo die con la capa de metal que lo interconecta

Fuente: K. Shirriff, A look at the die of the 8086 processor, 2020

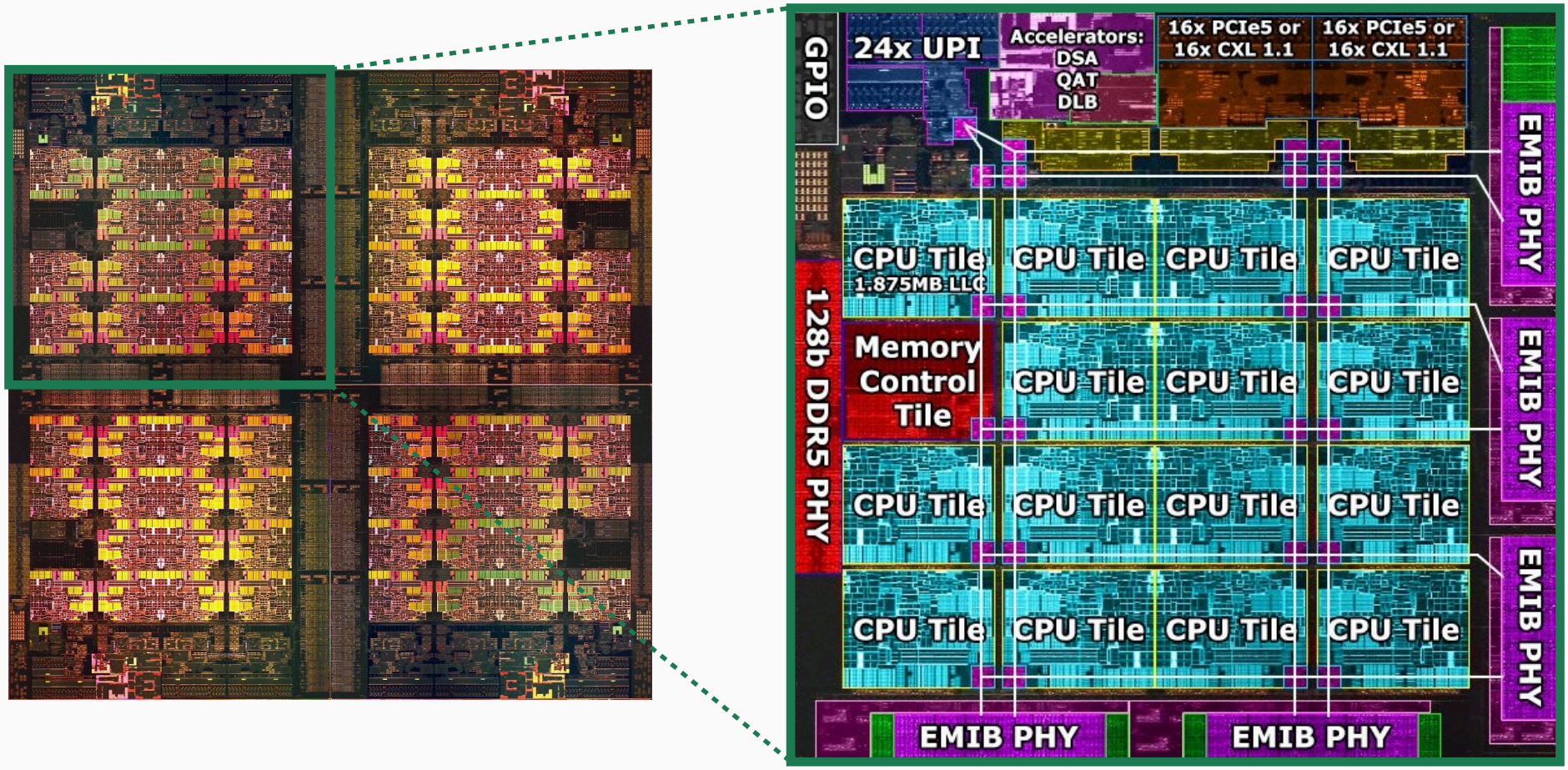
Von Neumann: no es un dibujo, es silicio



Intel Xeon “Sapphire Rapids”: 4 tiles unidos por EMIB, Intel 7

Fuente: TechPowerUp, Intel Sapphire Rapids Xeon 4-tile MCM Annotated, 2022

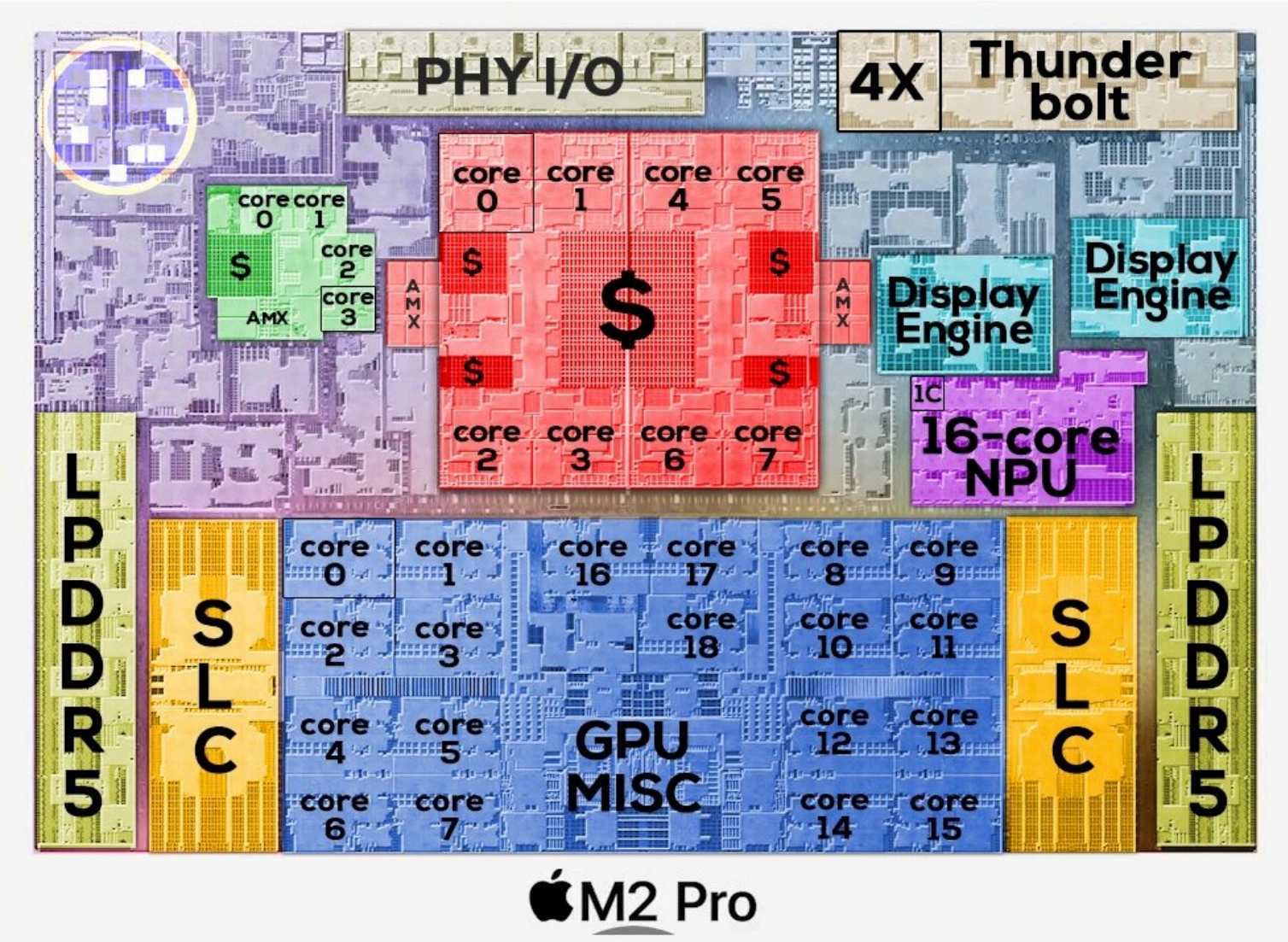
Von Neumann: no es un dibujo, es silicio



Uno de los 4 tiles: 15 cores, controlador de memoria, PCIe y aceleradores

Fuente: TechPowerUp, Intel Sapphire Rapids Xeon 4-tile MCM Annotated, 2022

Von Neumann: no es un dibujo, es silicio



TSMC N5P, 2023

Fuente: High Yield, M2 Pro die shot annotation, 2023 · youtube.com/@HighYield

Mismo modelo, ¿mismo idioma?

- A nivel arquitectural, el 8086 (1978), los cores del Xeon Sapphire Rapids (2023) y los cores del Apple M2 Pro (2023) siguen el modelo de Von Neumann
- Un programa escrito para el 8086 en 1978 todavía corre en el Sapphire Rapids, pero no en el M2

Mismo modelo, ¿mismo idioma?

- A nivel arquitectural, el 8086 (1978), los cores del Xeon Sapphire Rapids (2023) y los cores del Apple M2 Pro (2023) siguen el modelo de Von Neumann
- Un programa escrito para el 8086 en 1978 todavía corre en el Sapphire Rapids, pero no en el M2

¿Qué hace que dos procesadores “hablen el mismo idioma”?

Mismo modelo, ¿mismo idioma?

- A nivel arquitectural, el 8086 (1978), los cores del Xeon Sapphire Rapids (2023) y los cores del Apple M2 Pro (2023) siguen el modelo de Von Neumann
- Un programa escrito para el 8086 en 1978 todavía corre en el Sapphire Rapids, pero no en el M2

¿Qué hace que dos procesadores “hablen el mismo idioma”?

La ISA

Instruction Set Architecture

ISA: definición

Definición y especificación

ISA: definición

> ***ISA:*** *la especificación de la interfaz entre los programas y el hardware que debe llevar a cabo el trabajo de esos programas.*

Patt & Patel, Introduction to Computing Systems, *3ra ed., 2020*

ISA: definición

> ***ISA:*** *la especificación de la interfaz entre los programas y el hardware que debe llevar a cabo el trabajo de esos programas.*

- La ISA es el límite entre el hardware y el software.
- Define un contrato para los programadores de la plataforma y cuáles son las operaciones definidas. Entre ellos:
  - Representación, almacenamiento y acceso a datos.
  - Operaciones disponibles.
  - Codificación de las operaciones.

Patt & Patel, Introduction to Computing Systems, *3ra ed., 2020*

ISA: definición

Al ser una especificación, puede tener múltiples implementaciones

ISA: definición

Al ser una especificación, puede tener múltiples implementaciones



la especificación



SiFive
Núcleos comerciales para licenciar



XiangShan
Open source de alto rendimiento (ICT-CAS)



Espressif
Microcontroladores ESP32-C3 y C6

ISA: definición

Al ser una especificación, puede tener múltiples implementaciones



SiFive
Núcleos comerciales para licenciar

XiangShan
Open source de alto rendimiento (ICT-CAS)

Espressif
Microcontroladores ESP32-C3 y C6

Las tres cumplen la misma especificación (cada una con sus respectivas extensiones) pero la implementación puede ser completamente distinta

ISA: la especificación

La ISA se publica como un manual de referencia: es el contrato que tienen que respetar tanto el hardware como el software

ISA	Manual	Páginas
x86-64	*Intel 64 and IA-32 Architectures Software Developer's Manual* (vol. 1 a 4, 2026)	5364
Arm A-profile	*Arm Architecture Reference Manual for A-profile architecture* (DDI 0487 M.c, 2026)	17 145
RISC-V	*The RISC-V Instruction Set Manual*: vol. I (no privilegiado) + vol. II (privilegiado), 2024	670 + 172



Definición y especificación

Instrucciones

ISA: instrucciones

> ***Instrucción:*** *la unidad más chica de trabajo que sabe hacer un procesador.*

ISA: instrucciones

> ***Instrucción:*** *la unidad más chica de trabajo que sabe hacer un procesador.*

`0001 0011 0001 0010`

lenguaje de máquina: lo que ejecuta el procesador



`ADD R3, R1, R2`

mnemónico: lo que escribe y entiende una persona

ISA: instrucciones

> ***Instrucción:*** *la unidad más chica de trabajo que sabe hacer un procesador.*

`0001 0011 0001 0010`

lenguaje de máquina: lo que ejecuta el procesador

$\leftrightarrow$

`ADD R3, R1, R2`

mnemónico: lo que escribe y entiende una persona

- **Sintaxis**: la forma en la que escribo las instrucciones. `ADD R3, R1, R2` se lee "sumá `R1` y `R2` y guardá el resultado en `R3`"
- **Semántica**: el efecto sobre el estado arquitectural:

$$R[3] \leftarrow R[1] + R[2]$$

ISA: instrucciones

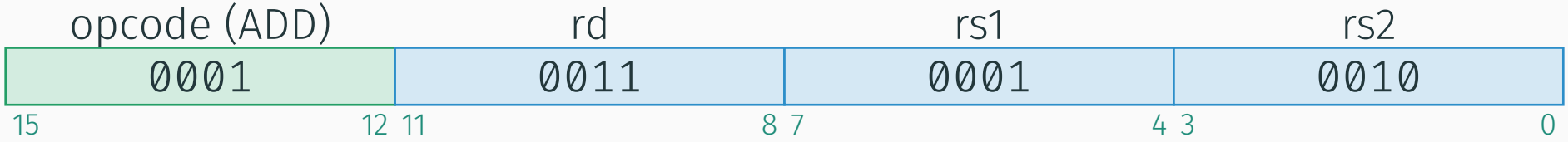
Una instrucción tiene dos partes:

- **Opcode**: define **qué hace**
- **Operandos**: definen **quiénes intervienen**. Pueden ser **registros**, ser un **inmediato** (una constante dentro de la instrucción) o un valor en **memoria**

ISA: instrucciones

Una instrucción tiene dos partes:

- **Opcode**: define **qué hace**
- **Operandos**: definen **quiénes intervienen**. Pueden ser **registros**, ser un **inmediato** (una constante dentro de la instrucción) o un valor en **memoria**

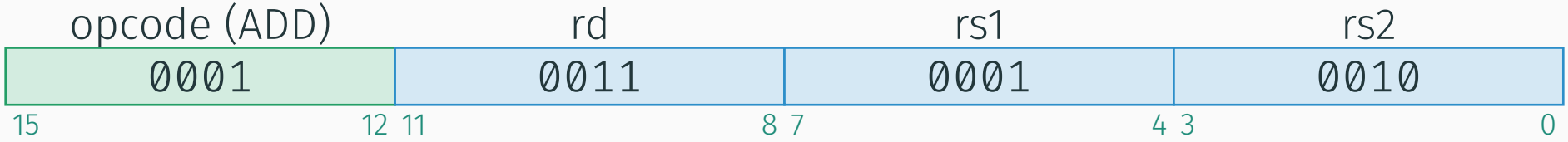


Un formato inventado de 16 bits: 4 bits de opcode y 4 por registro (16 registros)

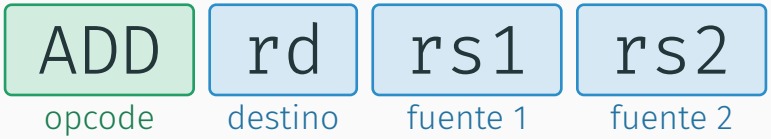
ISA: instrucciones

Una instrucción tiene dos partes:

- **Opcode**: define **qué hace**
- **Operandos**: definen **quiénes intervienen**. Pueden ser **registros**, ser un **inmediato** (una constante dentro de la instrucción) o un valor en **memoria**



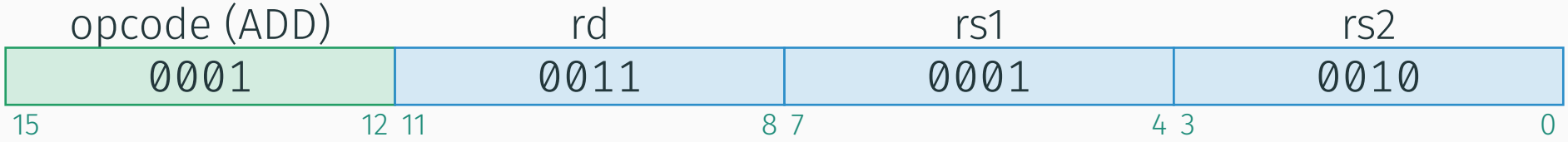
Un formato inventado de 16 bits: 4 bits de opcode y 4 por registro (16 registros)



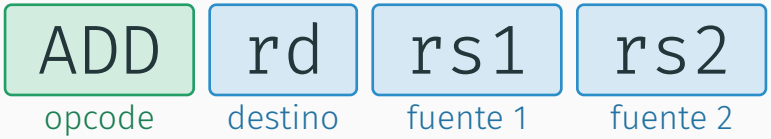
ISA: instrucciones

Una instrucción tiene dos partes:

- **Opcode**: define **qué hace**
- **Operandos**: definen **quiénes intervienen**. Pueden ser **registros**, ser un **inmediato** (una constante dentro de la instrucción) o un valor en **memoria**



Un formato inventado de 16 bits: 4 bits de opcode y 4 por registro (16 registros)



- **Tamaño fijo**: todas ocupan lo mismo (RISC-V: 32 bits). La siguiente está en `PC + 4` y los campos están siempre en el mismo lugar
- **Tamaño variable**: cada una ocupa los bytes que necesita (x86: de 1 a 15 bytes). Código más compacto, pero hay que decodificar una para saber dónde empieza la siguiente

ISA: tipos de instrucciones

Tipo de operación	Ejemplos
Aritméticas y lógicas	Suma, resta, and, or, multiplicación, división
Transferencia de datos	Loads y stores (moves en máquinas con direccionamiento a memoria)
Control	Saltos condicionales e incondicionales, llamada y retorno de funciones, traps
Sistema	Llamadas al sistema operativo, manejo de memoria virtual
Punto flotante	Suma, multiplicación, división y comparación en punto flotante
Decimal	Suma y multiplicación decimal, conversiones de decimal a caracteres
Strings	Mover, comparar y buscar strings
Gráficos	Operaciones sobre píxeles y vértices, compresión y descompresión

Hennessy & Patterson, Computer Architecture: A Quantitative Approach*, 6ta ed., 2017, apéndice A*

ISA: tipos de instrucciones

Tipo de operación	Ejemplos
Aritméticas y lógicas	Suma, resta, and, or, multiplicación, división
Transferencia de datos	Loads y stores (moves en máquinas con direccionamiento a memoria)
Control	Saltos condicionales e incondicionales, llamada y retorno de funciones, traps
Sistema	Llamadas al sistema operativo, manejo de memoria virtual
Punto flotante	Suma, multiplicación, división y comparación en punto flotante
Decimal	Suma y multiplicación decimal, conversiones de decimal a caracteres
Strings	Mover, comparar y buscar strings
Gráficos	Operaciones sobre píxeles y vértices, compresión y descompresión

Vamos a ver estas instrucciones usando un pseudo-assembler

Hennessy & Patterson, Computer Architecture: A Quantitative Approach*, 6ta ed., 2017, apéndice A*

ISA: aritméticas y lógicas

Instrucción	Efecto
`ADD rd, rs1, rs2`	`R[rd] ← R[rs1] + R[rs2]`
`SUB rd, rs1, rs2`	`R[rd] ← R[rs1] - R[rs2]`
`AND rd, rs1, rs2`	`R[rd] ← R[rs1] & R[rs2]` (bit a bit)
`SLLI rd, rs1, imm`	`R[rd] ← R[rs1] << imm`
`ADDI rd, rs1, imm`	`R[rd] ← R[rs1] + imm`

ISA: aritméticas y lógicas

Instrucción	Efecto
`ADD rd, rs1, rs2`	`R[rd] ← R[rs1] + R[rs2]`
`SUB rd, rs1, rs2`	`R[rd] ← R[rs1] - R[rs2]`
`AND rd, rs1, rs2`	`R[rd] ← R[rs1] & R[rs2]` (bit a bit)
`SLLI rd, rs1, imm`	`R[rd] ← R[rs1] << imm`
`ADDI rd, rs1, imm`	`R[rd] ← R[rs1] + imm`

- Las ejecuta la ALU, sobre registros e inmediatos

ISA: transferencia de datos

Instrucción	Efecto
`LOAD rd, imm(rs1)`	`R[rd] ← Mem[R[rs1] + imm]`
`STORE rs2, imm(rs1)`	`Mem[R[rs1] + imm] ← R[rs2]`
`MOV rd, rs1`	`R[rd] ← R[rs1]`
`LI rd, imm`	`R[rd] ← imm`

ISA: transferencia de datos

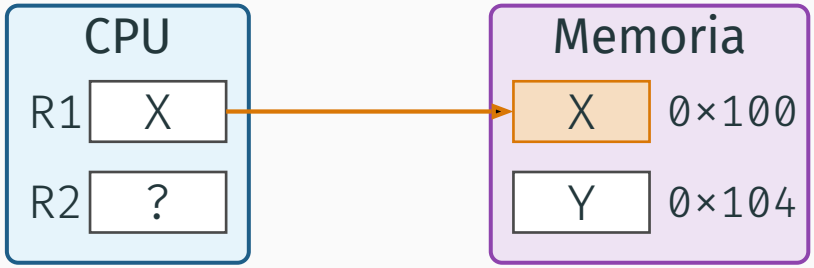
Instrucción	Efecto
`LOAD rd, imm(rs1)`	`R[rd] ← Mem[R[rs1] + imm]`
`STORE rs2, imm(rs1)`	`Mem[R[rs1] + imm] ← R[rs2]`
`MOV rd, rs1`	`R[rd] ← R[rs1]`
`LI rd, imm`	`R[rd] ← imm`

- `LOAD` y `STORE` mueven datos entre registros y memoria.
- `MOV` copia entre registros y `LI` (*load immediate*) carga una constante: ninguna toca la memoria

ISA: transferencia de datos

Mismo estado inicial: `R1` tiene el valor `X` y la dirección `0×104` tiene el valor `Y`

STORE: de registro a memoria



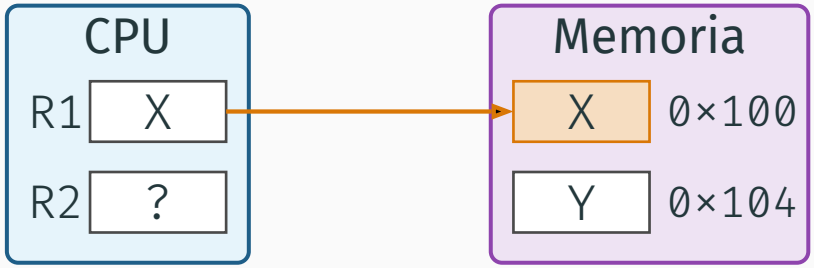

```
STORE R1, [0×100]
```

Mem[0x100] ← R1: la memoria pasa a tener X

ISA: transferencia de datos

Mismo estado inicial: `R1` tiene el valor `X` y la dirección `0×104` tiene el valor `Y`

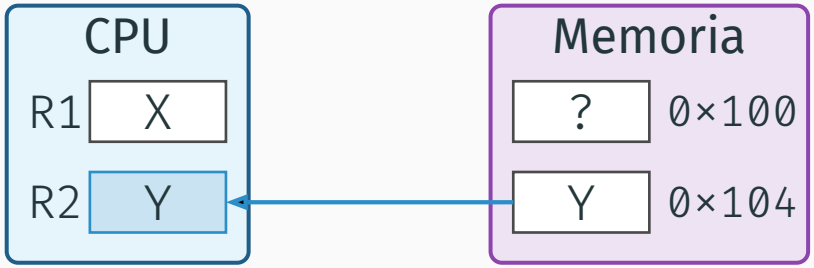
STORE: de registro a memoria




```
STORE R1, [0×100]
```

Mem[0x100] ← R1: la memoria pasa a tener X

LOAD: de memoria a registro




```
LOAD R2, [0×104]
```

R2 ← Mem[0x104]: el registro pasa a tener Y

ISA: control de flujo

Un **salto** modifica el PC

Instrucción	Efecto
`BEQ rs1, rs2, desp`	`PC ← (R[rs1] = R[rs2]) ? PC + desp : PC + 4`
`BGE rs1, rs2, desp`	`PC ← (R[rs1] ≥ R[rs2]) ? PC + desp : PC + 4`
`JUMP desp`	`PC ← PC + desp`
`JAL desp`	`R[ra] ← PC + 4;  PC ← PC + desp`
`RET`	`PC ← R[ra]`

ISA: control de flujo

Un **salto** modifica el PC

Instrucción	Efecto
`BEQ rs1, rs2, desp`	`PC ← (R[rs1] = R[rs2]) ? PC + desp : PC + 4`
`BGE rs1, rs2, desp`	`PC ← (R[rs1] ≥ R[rs2]) ? PC + desp : PC + 4`
`JUMP desp`	`PC ← PC + desp`
`JAL desp`	`R[ra] ← PC + 4;  PC ← PC + desp`
`RET`	`PC ← R[ra]`

- **Condicionales** (*branches*): comparan dos registros y deciden si saltan
- **Incondicionales** (*jumps*): saltan siempre. `JAL` guarda en `ra` (el registro de retorno) adónde volver, y `RET` vuelve ahí: así se llama a una función

ISA: control de flujo

Hay dos formas de comparar y saltar:

Con flags (x86, ARM)

```
cmp eax, ebx
jl  menor
```

si eax < ebx, salta a menor

`cmp` compara y guarda el resultado en los **flags** (negativo, cero, carry, overflow). Después el salto mira los flags

ISA: control de flujo

Hay dos formas de comparar y saltar:

Con flags (x86, ARM)

```
cmp eax, ebx
jl  menor
```

si eax < ebx, salta a menor

`cmp` compara y guarda el resultado en los **flags** (negativo, cero, carry, overflow). Después el salto mira los flags

Sin flags (RISC-V)

```
blt a0, a1, menor
```

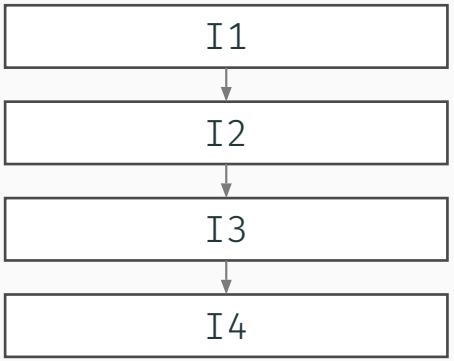
si a0 < a1, salta a menor

Compara y salta en una sola instrucción

ISA: control de flujo

Toda decisión de un programa (`if`, `while`, `for`, `switch`) se implementa con saltos:

Secuencial

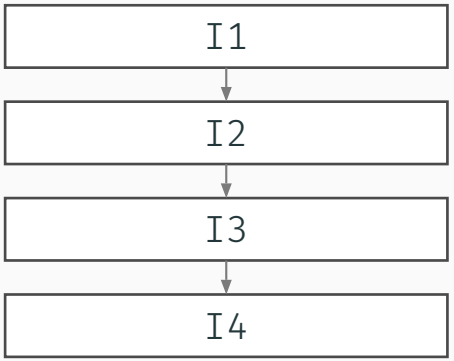



```
a; b; c; d;
```

ISA: control de flujo

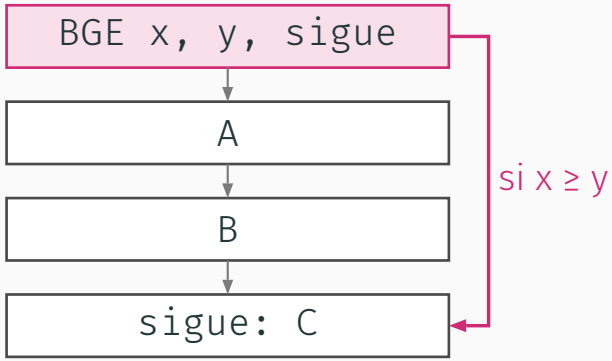
Toda decisión de un programa (`if`, `while`, `for`, `switch`) se implementa con saltos:

Secuencial




```
a; b; c; d;
```

Condicional

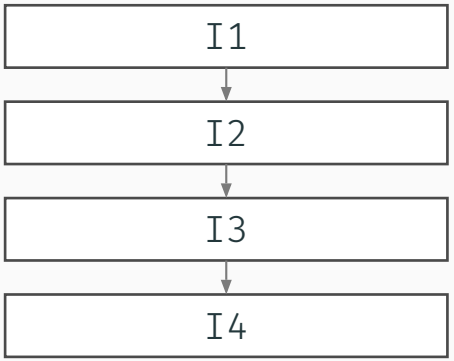



```
if (x < y) { A; B; } C;
```

ISA: control de flujo

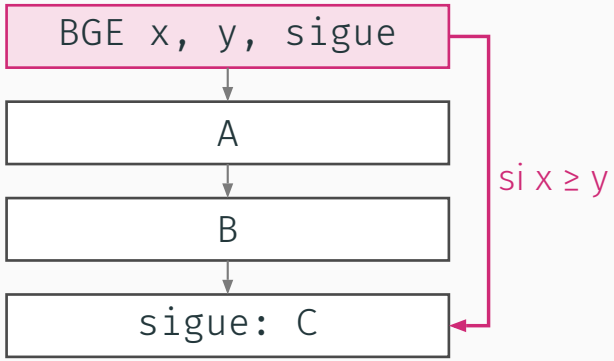
Toda decisión de un programa (`if`, `while`, `for`, `switch`) se implementa con saltos:

Secuencial



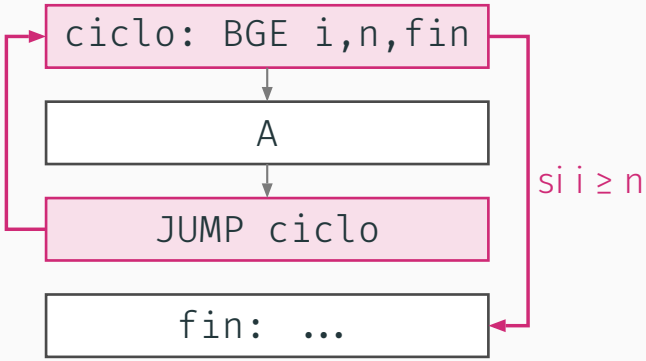

```
a; b; c; d;
```

Condicional




```
if (x < y) { A; B; } C;
```

Ciclos

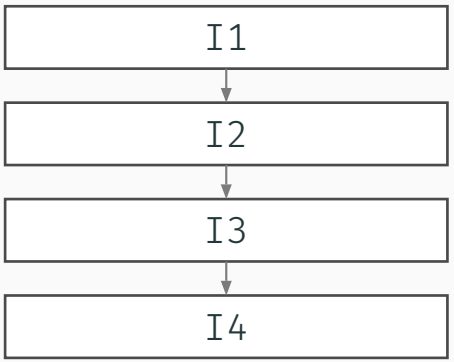



```
while (i < n) { A; }
```

ISA: control de flujo

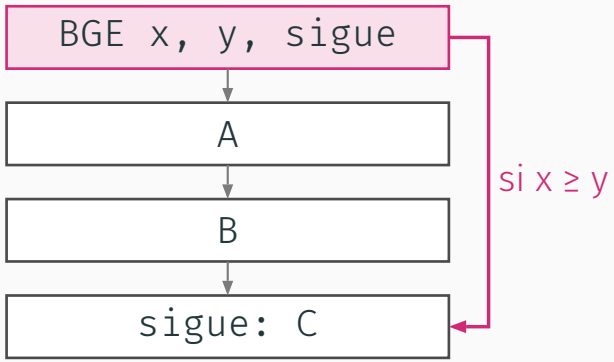
Toda decisión de un programa (`if`, `while`, `for`, `switch`) se implementa con saltos:

Secuencial



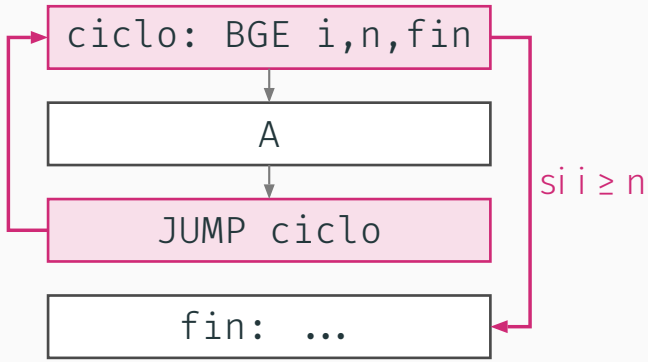

```
a; b; c; d;
```

Condicional




```
if (x < y) { A; B; } C;
```

Ciclos




```
while (i < n) { A; }
```

Las **etiquetas** (`sigue`, `ciclo`, `fin`) son un **alias** de una dirección de memoria: el nombre de la dirección donde está esa instrucción. El ensamblador las reemplaza por la dirección (o por la distancia hasta ella)

ISA: registros



Definición y especificación

Instrucciones

Registros y palabra

ISA: registros

> ***Registros:*** *memoria de acceso rápido que usa el procesador para guardar valores.*

ISA: registros

> ***Registros:*** *memoria de acceso rápido que usa el procesador para guardar valores.*

- La ISA define **cuántos** hay y qué tamaño tienen. Las variables que se usan seguido viven en registros, para evitar ir a memoria

ISA: registros

> ***Registros:*** *memoria de acceso rápido que usa el procesador para guardar valores.*

- La ISA define **cuántos** hay y qué tamaño tienen. Las variables que se usan seguido viven en registros, para evitar ir a memoria

Pocos registros: instrucciones más cortas, menos estado que guardar al llamar a una función o cambiar de proceso, banco más chico (útil en embebidos)

Muchos registros: menos accesos a memoria y más libertad para el compilador

ISA	Registros enteros
x86-32	8
x86-64	16
ARMv8-A	31
RISC-V RV32I	32
RISC-V RV32E	16

ISA: la palabra

> ***Palabra*** *(word size): la unidad de dato natural de la arquitectura*

ISA: la palabra

> ***Palabra*** *(word size): la unidad de dato natural de la arquitectura*

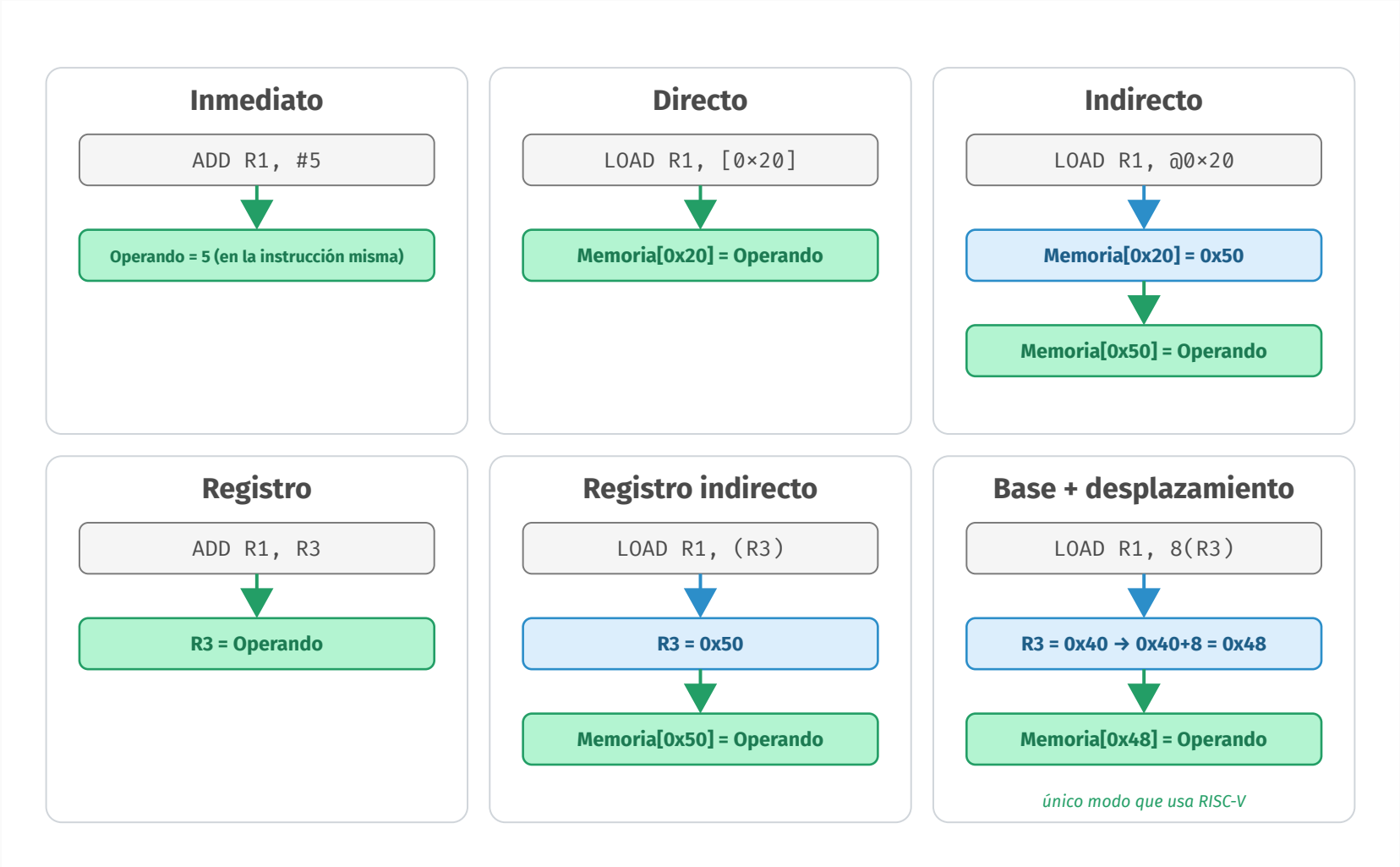
- Definida por el tamaño de los registros
- Acota el rango de los enteros que se operan en una instrucción

ISA: modos de direccionamiento

- Definición y especificación
- Instrucciones
- Registros y palabra
- **Modos de direccionamiento**

ISA: modos de direccionamiento

Un modo de direccionamiento define **dónde** está el operando.



ISA: modos de direccionamiento

Tres ejemplos en x86 (32 bits):

Inmediato

el operando viaja en la instrucción

```
mov eax, 5
```

eax = 5



ISA: modos de direccionamiento

Tres ejemplos en x86 (32 bits):

Inmediato

el operando viaja en la instrucción

Registro

el operando está en un registro

```
mov eax, 5
```

eax = 5

```
add eax, ebx
```

eax = eax + ebx

B8 05 00 00 00

01 D8

ISA: modos de direccionamiento

Tres ejemplos en x86 (32 bits):

Inmediato

el operando viaja en la instrucción

Registro

el operando está en un registro

Memoria

el operando está en `Mem[ebx + 8]`

```
mov eax, 5
```

eax = 5

B8 05 00 00 00

```
add eax, ebx
```

eax = eax + ebx

01 D8

```
mov eax, [ebx+8]
```

eax = Mem[ebx + 8]

8B 43 08

ISA: modos de direccionamiento

Tres ejemplos en x86 (32 bits):

Inmediato

el operando viaja en la instrucción

Registro

el operando está en un registro

Memoria

el operando está en `Mem[ebx + 8]`

`mov eax, 5`

eax = 5

`B8 05 00 00 00`

`add eax, ebx`

eax = eax + ebx

`01 D8`

`mov eax, [ebx+8]`

eax = Mem[ebx + 8]

`8B 43 08`

opcode ModRM: qué registros y qué modo desplazamiento inmediato o

ISA: modos de direccionamiento

Tres ejemplos en x86 (32 bits):

Inmediato

el operando viaja en la instrucción

`mov eax, 5`

eax = 5

`B8 05 00 00 00`

Registro

el operando está en un registro

`add eax, ebx`

eax = eax + ebx

`01 D8`

Memoria

el operando está en `Mem[ebx + 8]`

`mov eax, [ebx+8]`

eax = Mem[ebx + 8]

`8B 43 08`

opcode ModRM: qué registros y qué modo desplazamiento inmediato o

A notar:

ISA: modos de direccionamiento

Tres ejemplos en x86 (32 bits):

Inmediato
el operando viaja en la instrucción

Registro
el operando está en un registro

Memoria
el operando está en `Mem[ebx + 8]`

`mov eax, 5`
eax = 5

`B8 05 00 00 00`

`add eax, ebx`
eax = eax + ebx

`01 D8`

`mov eax, [ebx+8]`
eax = Mem[ebx + 8]

`8B 43 08`

opcode ModRM: qué registros y qué modo desplazamiento inmediato o

A notar:

- Tres instrucciones, tres largos distintos (5, 2 y 3 bytes): x86 es de tamaño variable

ISA: modos de direccionamiento

Tres ejemplos en x86 (32 bits):

Inmediato
el operando viaja en la instrucción

Registro
el operando está en un registro

Memoria
el operando está en `Mem[ebx + 8]`

`mov eax, 5`
eax = 5

`B8 05 00 00 00`

`add eax, ebx`
eax = eax + ebx

`01 D8`

`mov eax, [ebx+8]`
eax = Mem[ebx + 8]

`8B 43 08`

opcode ModRM: qué registros y qué modo inmediato o desplazamiento

A notar:

- Tres instrucciones, tres largos distintos (5, 2 y 3 bytes): x86 es de tamaño variable
- ¿Por qué el 5 se guarda como `05 00 00 00`?

ISA: endianness

- Definición y especificación
- Instrucciones
- Registros y palabra
- Modos de direccionamiento
- **Endianness**

ISA: endianness

Endianness: ¿en qué orden se guardan en memoria los bytes de un dato de varios bytes?

Valor de 32 bits a guardar: 0x12345678

12	34	56	78
MSB			LSB

Big-Endian

Dirección	Byte
0×00	
0×01	
0×02	
0×03	

Little-Endian

Dirección	Byte
0×00	
0×01	
0×02	
0×03	

Dos formas posibles de acomodar los 4 bytes en memoria. ¿Cuál usamos?

ISA: endianness

Endianness: ¿en qué orden se guardan en memoria los bytes de un dato de varios bytes?

Valor de 32 bits a guardar: 0x12345678

12	34	56	78
MSB			LSB

Big-Endian

Dirección	Byte
0×00	12
0×01	
0×02	
0×03	

Little-Endian

Dirección	Byte
0×00	
0×01	
0×02	
0×03	

Big-Endian: 0x00 ← 12 (empezamos por el byte más significativo)

ISA: endianness

Endianness: ¿en qué orden se guardan en memoria los bytes de un dato de varios bytes?

Valor de 32 bits a guardar: 0x12345678

12	34	56	78
MSB			LSB

Big-Endian

Dirección	Byte
0×00	12
0×01	34
0×02	
0×03	

Little-Endian

Dirección	Byte
0×00	
0×01	
0×02	
0×03	

Big-Endian: 0x01 ← 34 (empezamos por el byte más significativo)

ISA: endianness

Endianness: ¿en qué orden se guardan en memoria los bytes de un dato de varios bytes?

Valor de 32 bits a guardar: 0x12345678

12	34	56	78
MSB			LSB

Big-Endian

Dirección	Byte
0×00	12
0×01	34
0×02	56
0×03	

Little-Endian

Dirección	Byte
0×00	
0×01	
0×02	
0×03	

Big-Endian: 0x02 ← 56 (empezamos por el byte más significativo)

ISA: endianness

Endianness: ¿en qué orden se guardan en memoria los bytes de un dato de varios bytes?

Valor de 32 bits a guardar: 0x12345678



Big-Endian

Dirección	Byte
0×00	12
0×01	34
0×02	56
0×03	78

Little-Endian

Dirección	Byte
0×00	
0×01	
0×02	
0×03	

Big-Endian: 0x03 ← 78 (empezamos por el byte más significativo)

ISA: endianness

Endianness: ¿en qué orden se guardan en memoria los bytes de un dato de varios bytes?

Valor de 32 bits a guardar: 0x12345678



Big-Endian

Dirección	Byte
0×00	12
0×01	34
0×02	56
0×03	78

Little-Endian

Dirección	Byte
0×00	
0×01	
0×02	
0×03	

Big-Endian completo. Ahora, Little-Endian:

ISA: endianness

Endianness: ¿en qué orden se guardan en memoria los bytes de un dato de varios bytes?

Valor de 32 bits a guardar: 0x12345678

12	34	56	78
MSB			LSB

Big-Endian

Dirección	Byte
0×00	12
0×01	34
0×02	56
0×03	78

Little-Endian

Dirección	Byte
0×00	78
0×01	
0×02	
0×03	

Little-Endian: 0x00 ← 78 (empezamos por el byte menos significativo)

ISA: endianness

Endianness: ¿en qué orden se guardan en memoria los bytes de un dato de varios bytes?

Valor de 32 bits a guardar: 0x12345678

12	34	56	78
MSB			LSB

Big-Endian

Dirección	Byte
0×00	12
0×01	34
0×02	56
0×03	78

Little-Endian

Dirección	Byte
0×00	78
0×01	56
0×02	
0×03	

Little-Endian: 0x01 ← 56 (empezamos por el byte menos significativo)

ISA: endianness

Endianness: ¿en qué orden se guardan en memoria los bytes de un dato de varios bytes?

Valor de 32 bits a guardar: 0x12345678

12	34	56	78
MSB			LSB

Big-Endian

Dirección	Byte
0×00	12
0×01	34
0×02	56
0×03	78

Little-Endian

Dirección	Byte
0×00	78
0×01	56
0×02	34
0×03	

Little-Endian: 0x02 ← 34 (empezamos por el byte menos significativo)

ISA: endianness

Endianness: ¿en qué orden se guardan en memoria los bytes de un dato de varios bytes?

Valor de 32 bits a guardar: 0x12345678



Big-Endian

Dirección	Byte
0×00	12
0×01	34
0×02	56
0×03	78

Little-Endian

Dirección	Byte
0×00	78
0×01	56
0×02	34
0×03	12

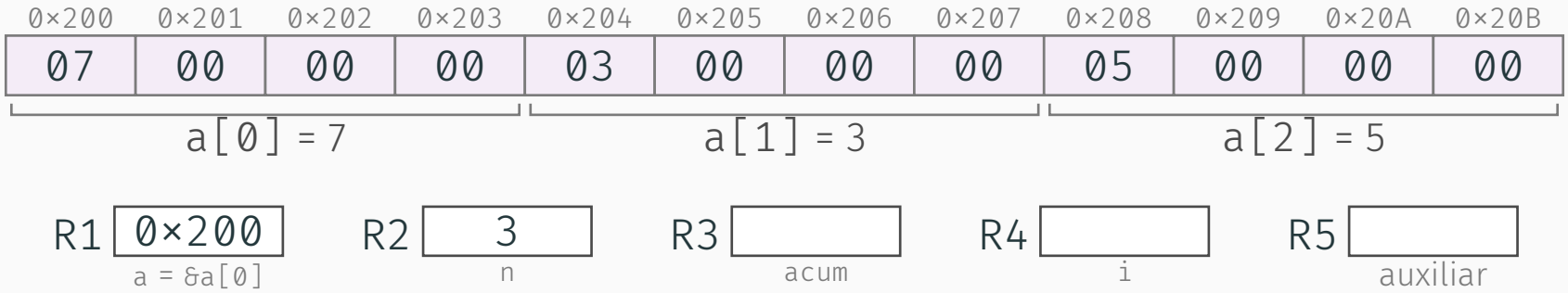
Little-Endian: 0x03 ← 12 (empezamos por el byte menos significativo)

ISA: de C a assembly

- Definición y especificación
- Instrucciones
- Registros y palabra
- Modos de direccionamiento
- Endianness
- **De C a assembly**

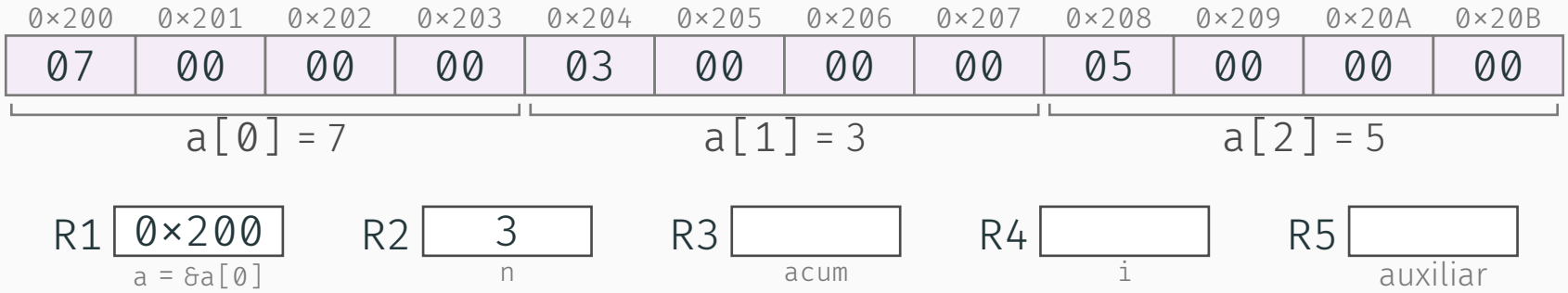
ISA: de C a assembly

Un ejemplo chiquito: `suma(a, 3)` con `a = {7, 3, 5}` guardado a partir de la dirección `0×200`



ISA: de C a assembly

Un ejemplo chiquito: `suma(a, 3)` con `a = {7, 3, 5}` guardado a partir de la dirección `0×200`



- La memoria se direcciona **por byte** y cada `int` ocupa **4 bytes** (*little-endian*): `a[0]` en `0×200`, `a[1]` en `0×204`, `a[2]` en `0×208`
- Entonces `a[i]` está en la dirección `a + 4·i`. En `R1` no está el arreglo, está la dirección donde empieza

ISA: de C a assembly

C

```c
int suma(int a[], int n) {
    int acum = 0;
    for (int i = 0; i < n; i++) {
        acum = acum + a[i];
    }
    return acum;
}
```

Assembly

```
suma:                   # R1 = a, R2 = n
    LI    R3, 0         # acum = 0
    LI    R4, 0         # i = 0
ciclo:
    BGE   R4, R2, fin   # si i ⩾ n, salir
    SLLI  R5, R4, 2     # R5 = i * 4
    ADD   R5, R1, R5    # R5 = &a[i]
    LOAD  R5, 0(R5)     # R5 = a[i]
    ADD   R3, R3, R5    # acum += a[i]
    ADDI  R4, R4, 1     # i++
    JUMP  ciclo         # volver a la condición
fin:
    MOV   R1, R3        # valor de retorno
    RET
```

Suma los `n` elementos del arreglo `a`, en un assembly genérico (no es de ninguna ISA real). Vamos bloque por bloque.

ISA: de C a assembly

C

```c
int suma(int a[], int n) {
    int acum = 0;
    for (int i = 0; i < n; i++) {
        acum = acum + a[i];
    }
    return acum;
}
```

Assembly

```
suma:                   # R1 = a, R2 = n
    LI    R3, 0         # acum = 0
    LI    R4, 0         # i = 0
ciclo:
    BGE   R4, R2, fin   # si i ⩾ n, salir
    SLLI  R5, R4, 2     # R5 = i * 4
    ADD   R5, R1, R5    # R5 = &a[i]
    LOAD  R5, 0(R5)     # R5 = a[i]
    ADD   R3, R3, R5    # acum += a[i]
    ADDI  R4, R4, 1     # i++
    JUMP  ciclo         # volver a la condición
fin:
    MOV   R1, R3        # valor de retorno
    RET
```

La función es una **etiqueta**: una dirección con nombre. Por convención de llamada, los argumentos llegan en `R1` (la dirección de `a`) y `R2` (`n`).

ISA: de C a assembly

C

```c
int suma(int a[], int n) {
    int acum = 0;
    for (int i = 0; i < n; i++) {
        acum = acum + a[i];
    }
    return acum;
}
```

Assembly

```
suma:                   # R1 = a, R2 = n
    LI    R3, 0         # acum = 0
    LI    R4, 0         # i = 0
ciclo:
    BGE   R4, R2, fin   # si i ⩾ n, salir
    SLLI  R5, R4, 2     # R5 = i * 4
    ADD   R5, R1, R5    # R5 = &a[i]
    LOAD  R5, 0(R5)     # R5 = a[i]
    ADD   R3, R3, R5    # acum += a[i]
    ADDI  R4, R4, 1     # i++
    JUMP  ciclo         # volver a la condición
fin:
    MOV   R1, R3        # valor de retorno
    RET
```

Las variables locales se asignan a registros: `acum` → `R3`, `i` → `R4`. `LI` carga una constante: direccionamiento **inmediato**.

ISA: de C a assembly

C

```c
int suma(int a[], int n) {
    int acum = 0;
    for (int i = 0; i < n; i++) {
        acum = acum + a[i];
    }
    return acum;
}
```

Assembly

```
suma:                   # R1 = a, R2 = n
    LI    R3, 0         # acum = 0
    LI    R4, 0         # i = 0
ciclo:
    BGE   R4, R2, fin   # si i ⩾ n, salir
    SLLI  R5, R4, 2     # R5 = i * 4
    ADD   R5, R1, R5    # R5 = &a[i]
    LOAD  R5, 0(R5)     # R5 = a[i]
    ADD   R3, R3, R5    # acum += a[i]
    ADDI  R4, R4, 1     # i++
    JUMP  ciclo         # volver a la condición
fin:
    MOV   R1, R3        # valor de retorno
    RET
```

La condición del `for` es un salto condicional al principio del ciclo, con la condición negada: si `i` ⩾ `n`, salta a `fin`.

ISA: de C a assembly

C

```c
int suma(int a[], int n) {
    int acum = 0;
    for (int i = 0; i < n; i++) {
        acum = acum + a[i];
    }
    return acum;
}
```

Assembly

```
suma:                   # R1 = a, R2 = n
    LI    R3, 0         # acum = 0
    LI    R4, 0         # i = 0
ciclo:
    BGE   R4, R2, fin   # si i ⩾ n, salir
    SLLI  R5, R4, 2     # R5 = i * 4
    ADD   R5, R1, R5    # R5 = &a[i]
    LOAD  R5, 0(R5)     # R5 = a[i]
    ADD   R3, R3, R5    # acum += a[i]
    ADDI  R4, R4, 1     # i++
    JUMP  ciclo         # volver a la condición
fin:
    MOV   R1, R3        # valor de retorno
    RET
```

`a[i]` está en la dirección `a + 4·i` (cada `int` ocupa 4 bytes): `SLLI` multiplica por 4, `ADD` suma la base y `LOAD` lee el elemento con direccionamiento **base + desplazamiento**.

ISA: de C a assembly

C

```c
int suma(int a[], int n) {
    int acum = 0;
    for (int i = 0; i < n; i++) {
        acum = acum + a[i];
    }
    return acum;
}
```

Assembly

```
suma:                   # R1 = a, R2 = n
    LI    R3, 0         # acum = 0
    LI    R4, 0         # i = 0
ciclo:
    BGE   R4, R2, fin   # si i ⩾ n, salir
    SLLI  R5, R4, 2     # R5 = i * 4
    ADD   R5, R1, R5    # R5 = &a[i]
    LOAD  R5, 0(R5)     # R5 = a[i]
    ADD   R3, R3, R5    # acum += a[i]
    ADDI  R4, R4, 1     # i++
    JUMP  ciclo         # volver a la condición
fin:
    MOV   R1, R3        # valor de retorno
    RET
```

La suma opera solo sobre registros (direccionamiento **por registro**): `acum` ya está en `R3` y `a[i]` en `R5`.

ISA: de C a assembly

C

```c
int suma(int a[], int n) {
    int acum = 0;
    for (int i = 0; i < n; i++) {
        acum = acum + a[i];
    }
    return acum;
}
```

Assembly

```
suma:                   # R1 = a, R2 = n
    LI    R3, 0         # acum = 0
    LI    R4, 0         # i = 0
ciclo:
    BGE   R4, R2, fin   # si i ⩾ n, salir
    SLLI  R5, R4, 2     # R5 = i * 4
    ADD   R5, R1, R5    # R5 = &a[i]
    LOAD  R5, 0(R5)     # R5 = a[i]
    ADD   R3, R3, R5    # acum += a[i]
    ADDI  R4, R4, 1     # i++
    JUMP  ciclo         # volver a la condición
fin:
    MOV   R1, R3        # valor de retorno
    RET
```

`i++` y un salto incondicional hacia atrás: `JUMP ciclo` vuelve a evaluar la condición.

ISA: de C a assembly

C

```c
int suma(int a[], int n) {
    int acum = 0;
    for (int i = 0; i < n; i++) {
        acum = acum + a[i];
    }
    return acum;
}
```

Assembly

```
suma:                   # R1 = a, R2 = n
    LI    R3, 0         # acum = 0
    LI    R4, 0         # i = 0
ciclo:
    BGE   R4, R2, fin   # si i ⩾ n, salir
    SLLI  R5, R4, 2     # R5 = i * 4
    ADD   R5, R1, R5    # R5 = &a[i]
    LOAD  R5, 0(R5)     # R5 = a[i]
    ADD   R3, R3, R5    # acum += a[i]
    ADDI  R4, R4, 1     # i++
    JUMP  ciclo         # volver a la condición
fin:
    MOV   R1, R3        # valor de retorno
    RET
```

El valor de retorno va en `R1`, por convención. `RET` salta a la dirección de retorno que guardó el `JAL` del llamador.

ISA: criterios de diseño

- Definición y especificación
- Instrucciones
- Registros y palabra
- Modos de direccionamiento
- Endianness
- De C a assembly
- **Decisiones de diseño: RISC vs CISC**

ISA: criterios de diseño

Diseñar una ISA implica tomar decisiones que después no se pueden volver atrás

ISA: criterios de diseño

Diseñar una ISA implica tomar decisiones que después no se pueden volver atrás

Qué hay que decidir

- Cantidad y complejidad de las instrucciones
- Tamaño de instrucción: ¿fijo o variable?
- Operandos por instrucción y cantidad de registros
- Memoria: ¿por byte o por palabra? ¿big o little endian?
- Modos de direccionamiento: ¿muchos, pocos o uno solo?

ISA: criterios de diseño

Diseñar una ISA implica tomar decisiones que después no se pueden volver atrás

Qué hay que decidir

- Cantidad y complejidad de las instrucciones
- Tamaño de instrucción: ¿fijo o variable?
- Operandos por instrucción y cantidad de registros
- Memoria: ¿por byte o por palabra? ¿big o little endian?
- Modos de direccionamiento: ¿muchos, pocos o uno solo?

Con qué criterios

- Costo y simplicidad del hardware
- Tamaño del programa (*code density*)
- Facilidad para programar y compilar
- Espacio para crecer
- Retrocompatibilidad

ISA: RISC vs CISC

Dos filosofías para responder a esas decisiones:

ISA: RISC vs CISC

Dos filosofías para responder a esas decisiones:

CISC (*Complex Instruction Set Computer*): x86, VAX, Motorola 68000

- Muchas instrucciones que realicen tareas complejas (copiar un string entero en una instrucción)
- Muchos modos de direccionamiento
- Tamaño variable de instrucciones: código compacto, pero difícil de decodificar

ISA: RISC vs CISC

Dos filosofías para responder a esas decisiones:

CISC (*Complex Instruction Set Computer*): x86, VAX, Motorola 68000

- Muchas instrucciones que realicen tareas complejas (copiar un string entero en una instrucción)
- Muchos modos de direccionamiento
- Tamaño variable de instrucciones: código compacto, pero difícil de decodificar

RISC (*Reduced Instruction Set Computer*): RISC-V, ARM, MIPS

- Pocas instrucciones y simples
- *Load/store*: solo los loads y stores acceden a memoria, el resto opera sobre registros
- Tamaño fijo de instrucciones y muchos registros

ISA: RISC vs CISC

Criterio	x86-64	ARMv8-A (AArch64)	RISC-V (RV64GC)
Filosofía	CISC	RISC (load/store)	RISC (load/store)
Longitud de instrucción	Variable (1–15 B)	Fija (32 bits)	Fija (32 b) + 16 b (ext. C)
Registros enteros	16	31 (+ SP/XZR)	32 (x0 = cero)
Palabra (registros)	64 bits	64 bits	64 bits (RV64)
Qué llama *word*	16 bits (herencia del 8086)	32 bits	32 bits
Flags (N, Z, C, V)	Sí	Sí	No: comparaciones a registro
Endianness	Little	Bi (usualmente LE)	Little
Modos de direccionamiento	Muchos (índice, escala, desp...)	Moderados (pre/post-indexado...)	Mínimos (solo base + desp.)
Apertura de la ISA	Propietaria (Intel/AMD)	Propietaria (con licencia)	Abierta

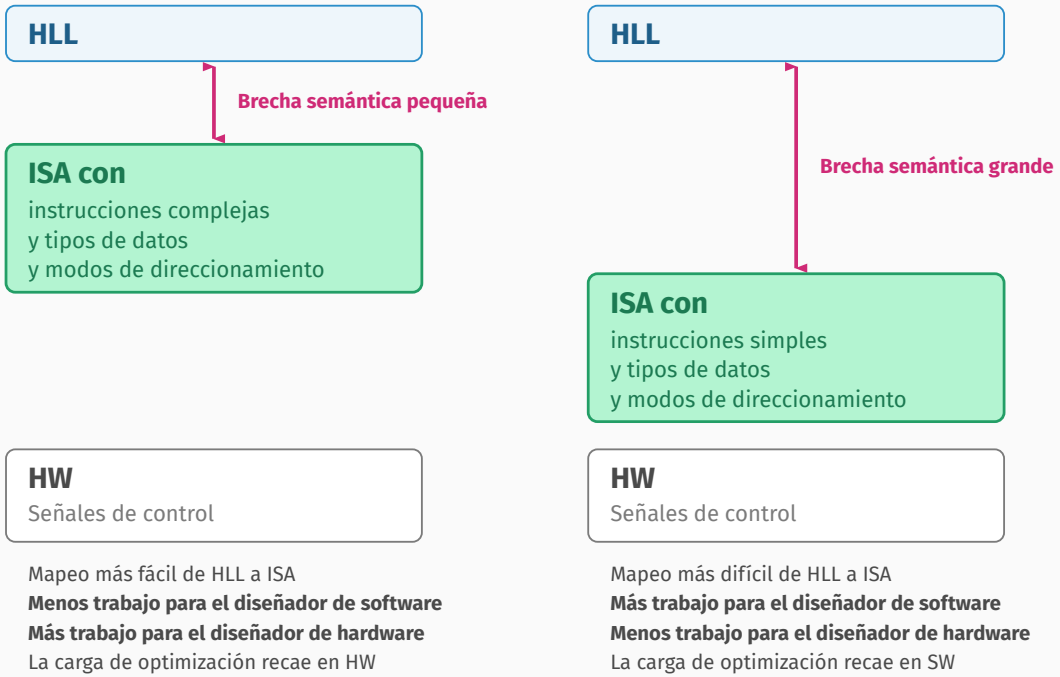
ISA: brecha semántica

Una ISA tiene que ser implementada. Según su complejidad, el trabajo recae en quien diseña el hardware o en quien escribe el compilador

ISA: brecha semántica

Una ISA tiene que ser implementada. Según su complejidad, el trabajo recae en quien diseña el hardware o en quien escribe el compilador

- Qué tan cerca están instrucciones, tipos de datos y modos de direccionamiento del HLL (lenguaje de alto nivel)



¿Cuánto trabajo queda del lado del hardware y cuánto del compilador?

Ciclo de instrucción

Ciclo de instrucción: definición

> ***Ciclo de instrucción:*** *la secuencia de pasos que realiza el procesador para procesar una instrucción.*

Ciclo de instrucción: definición

> ***Ciclo de instrucción:*** *la secuencia de pasos que realiza el procesador para procesar una instrucción.*

- Se divide en etapas, y la cantidad la define la **microarquitectura**. Para la materia vamos a ver **5 etapas**
- Se modela como una **máquina de estados**.

Patt & Patel, Introduction to Computing Systems, *3ra ed., 2020, capítulo 4*

Ciclo de instrucción: etapas



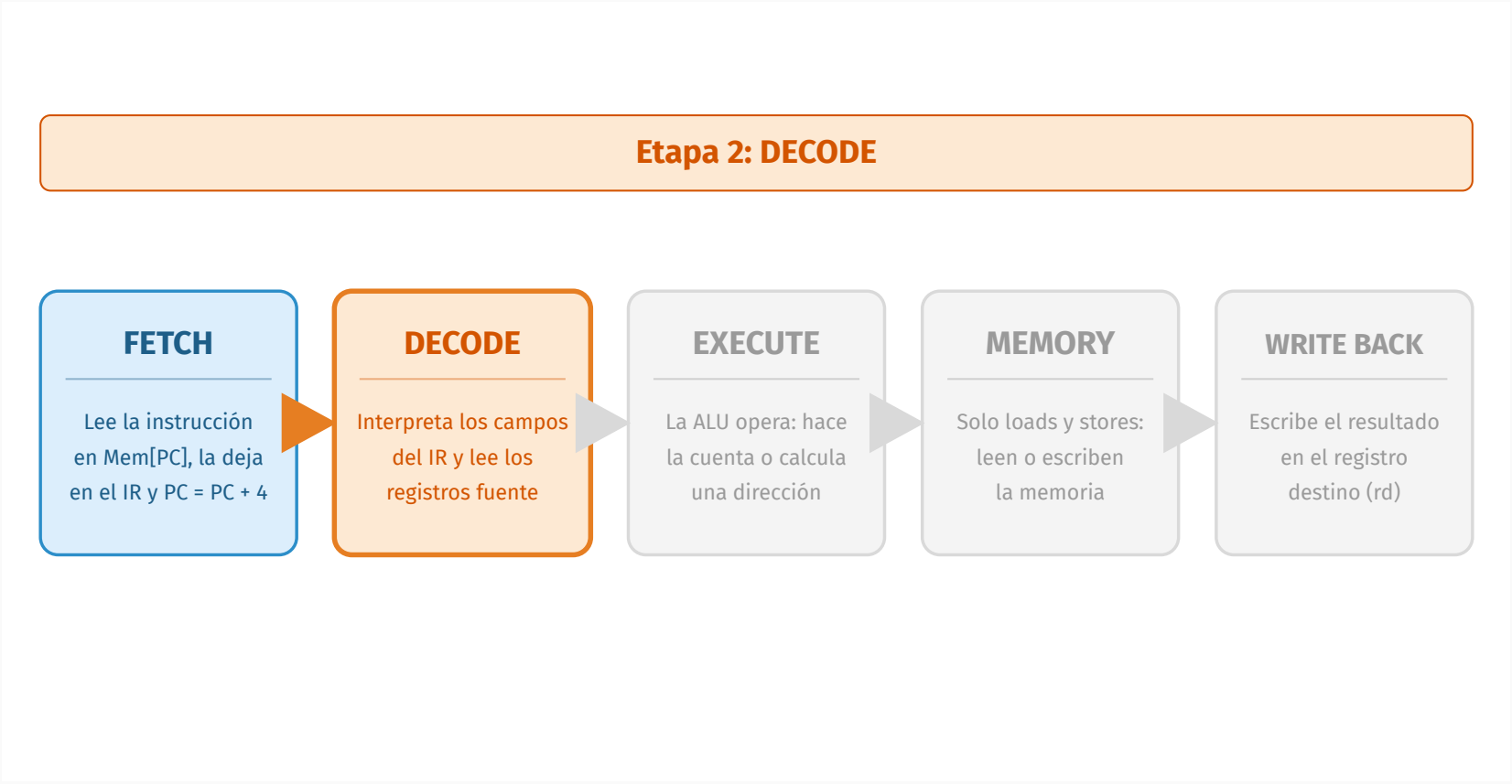
> Cada instrucción recorre las cinco etapas en orden, y al terminar arranca la siguiente

Ciclo de instrucción: etapas



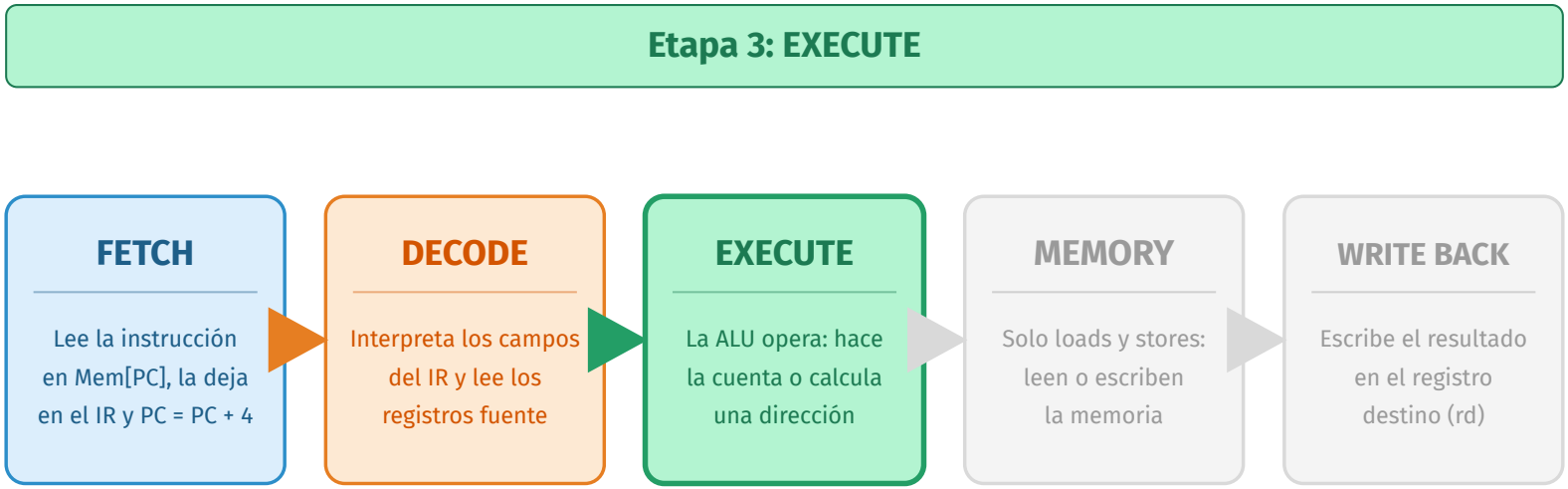
Fetch (IF): se lee de memoria la instrucción que apunta el **PC**, se guarda en el **IR** y el PC avanza a la siguiente (`PC + 4` con instrucciones de 32 bits)

Ciclo de instrucción: etapas



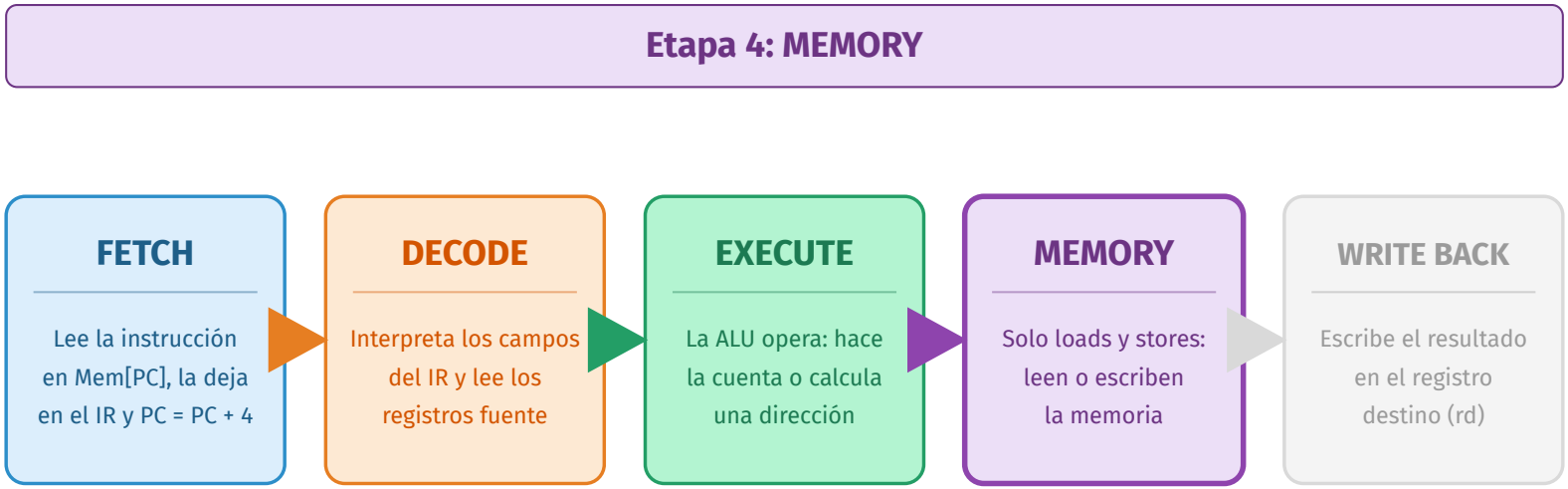
Decode (ID): se interpretan los campos del IR (qué operación y qué operandos) y se leen los registros fuente

Ciclo de instrucción: etapas



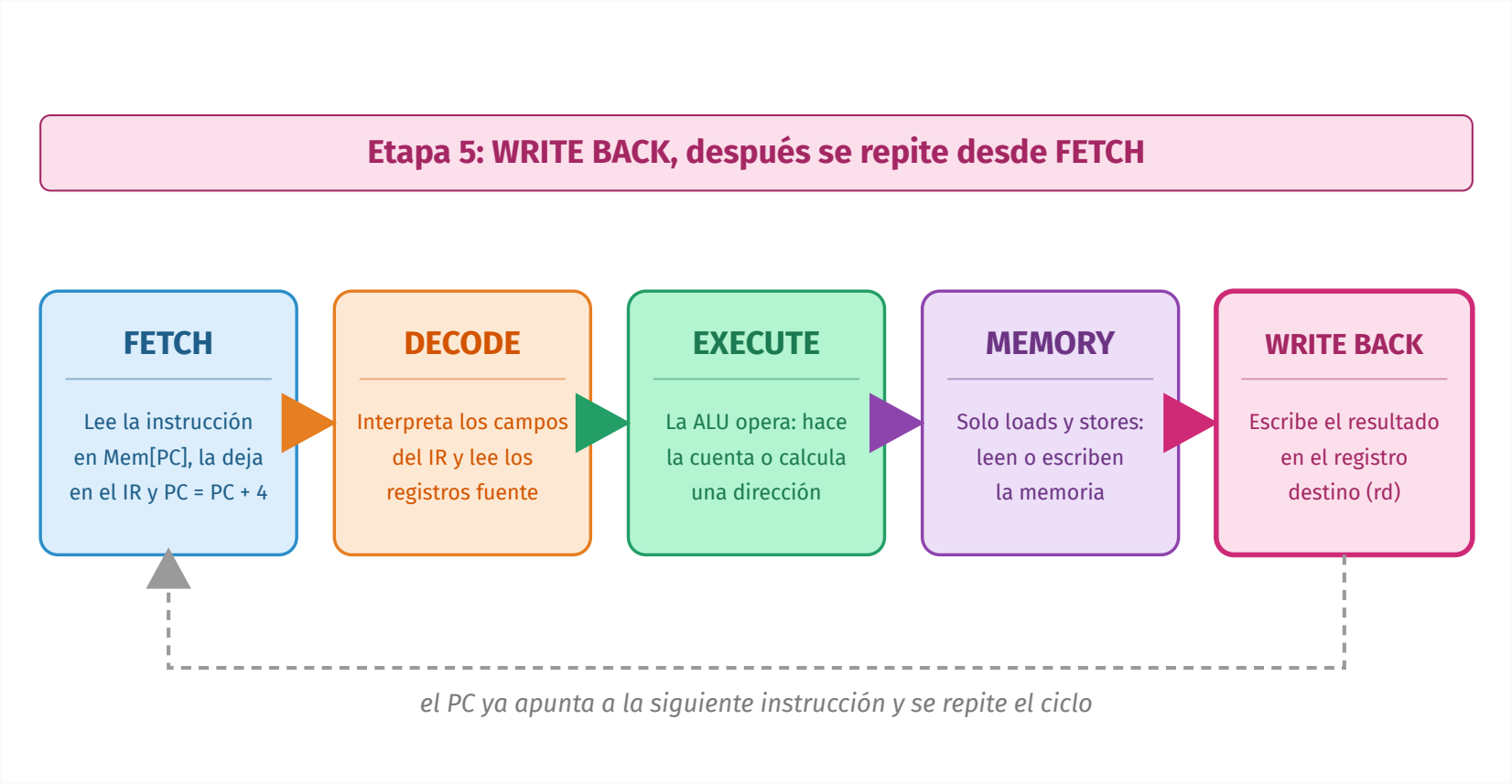
Execute (EX): la ALU hace la operación, calcula una dirección (loads y stores) o compara (saltos)

Ciclo de instrucción: etapas



Memory (MEM): solo los loads y stores leen o escriben la memoria

Ciclo de instrucción: etapas



Write back (WB): el resultado se escribe en el registro destino. Después se vuelve a fetch

Ciclo de instrucción: cómo ejecuta un programa

C	Assembly

`c = a + b;`	`LOAD  R1, [0×100]  # R1 = a`
	`LOAD  R2, [0×104]  # R2 = b`
	`ADD   R3, R1, R2   # R3 = a + b`
	`STORE R3, [0×108]  # c = R3`

Ciclo de instrucción: cómo ejecuta un programa

C	Assembly

`c = a + b;`	`LOAD  R1, [0×100]  # R1 = a`
	`LOAD  R2, [0×104]  # R2 = b`
	`ADD   R3, R1, R2   # R3 = a + b`
	`STORE R3, [0×108]  # c = R3`

- El programa está en memoria desde la dirección `0×00`; `a`, `b` y `c` en `0×100`, `0×104` y `0×108`
- `[0×100]`: direccionamiento directo, la dirección del dato viene en la instrucción

Ciclo de instrucción: cómo ejecuta un programa

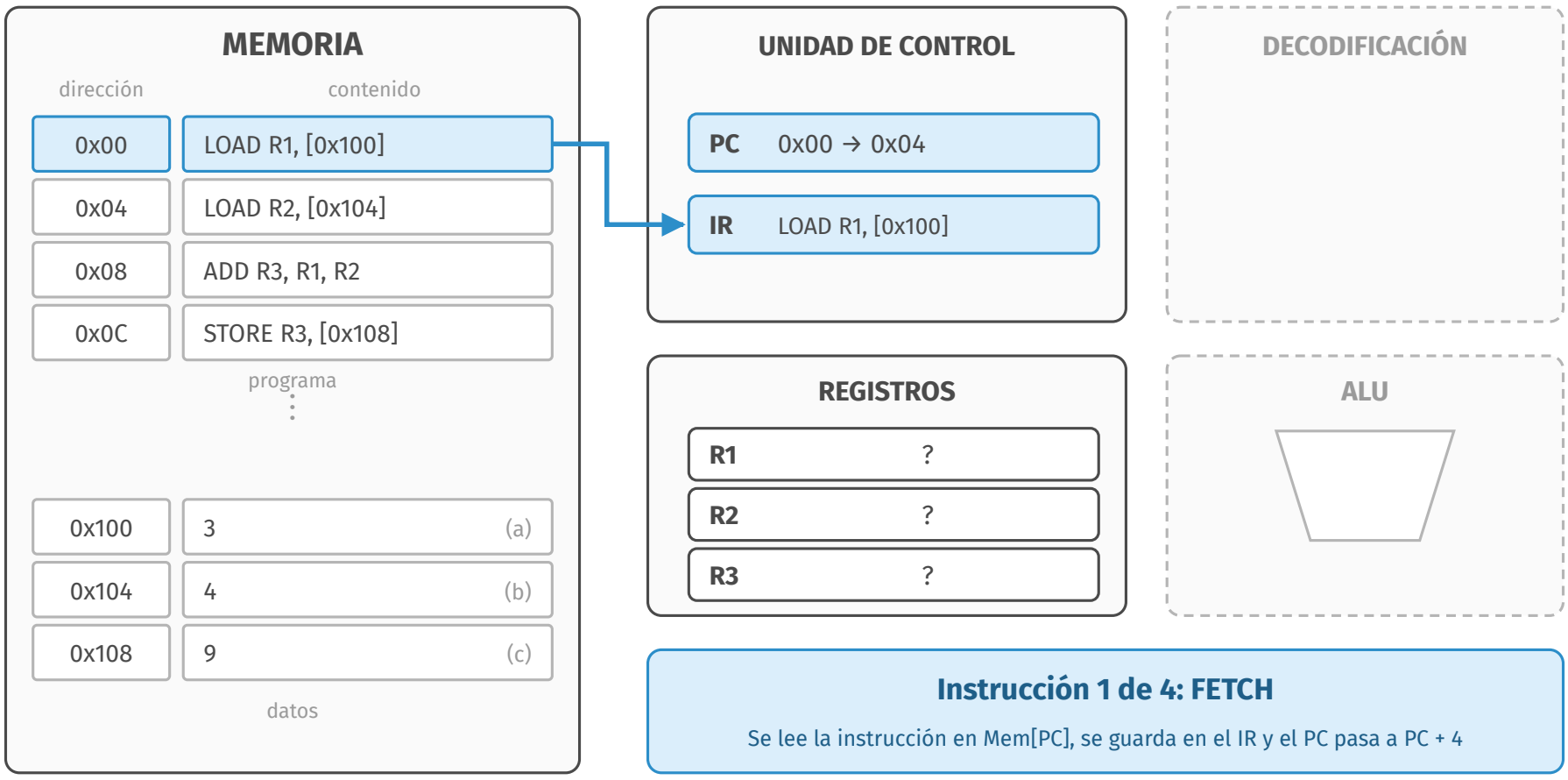


Estado inicial

El programa y los datos están en la misma memoria

Ciclo de instrucción: cómo ejecuta un programa

DEPARTAMENTO DE COMPUTACION
Facultad de Ciencias Exactas y Naturales - UBA



Ciclo de instrucción: cómo ejecuta un programa

DEPARTAMENTO DE COMPUTACION
Facultad de Ciencias Exactas y Naturales - UBA

MEMORIA

dirección	contenido	
0x00	LOAD R1, [0x100]	
0x04	LOAD R2, [0x104]	
0x08	ADD R3, R1, R2	
0x0C	STORE R3, [0x108]	

programa

⋮

0x100	3	(a)
0x104	4	(b)
0x108	9	(c)

datos

UNIDAD DE CONTROL

PC	0x04
IR	LOAD R1, [0x100]

DECODIFICACIÓN

LOAD (lectura de memoria)

rd = R1

dirección = 0x100

REGISTROS

R1	?
R2	?
R3	?

ALU



Instrucción 1 de 4: DECODE

Es un load: la dirección del dato viene en la misma instrucción

Ciclo de instrucción: cómo ejecuta un programa

DEPARTAMENTO DE COMPUTACION
Facultad de Ciencias Exactas y Naturales - UBA

MEMORIA

dirección	contenido
0x00	LOAD R1, [0x100]
0x04	LOAD R2, [0x104]
0x08	ADD R3, R1, R2
0x0C	STORE R3, [0x108]

programa

0x100	3	(a)
0x104	4	(b)
0x108	9	(c)

datos

UNIDAD DE CONTROL

PC	0x04
IR	LOAD R1, [0x100]

REGISTROS

R1	?
R2	?
R3	?

DECODIFICACIÓN

LOAD (lectura de memoria)

rd = R1

dirección = 0x100

ALU



dirección = 0x100

Instrucción 1 de 4: EXECUTE

Se resuelve la dirección del dato: 0x100

Ciclo de instrucción: cómo ejecuta un programa

DEPARTAMENTO DE COMPUTACION
Facultad de Ciencias Exactas y Naturales - UBA

MEMORIA

dirección	contenido	
0x00	LOAD R1, [0x100]	
0x04	LOAD R2, [0x104]	
0x08	ADD R3, R1, R2	
0x0C	STORE R3, [0x108]	

programa

⋮

0x100	**3**	(a)
0x104	4	(b)
0x108	9	(c)

datos

UNIDAD DE CONTROL

PC	0x04
IR	LOAD R1, [0x100]

REGISTROS

R1	?
R2	?
R3	?

DECODIFICACIÓN

LOAD (lectura de memoria)

rd = R1

dirección = 0x100

ALU

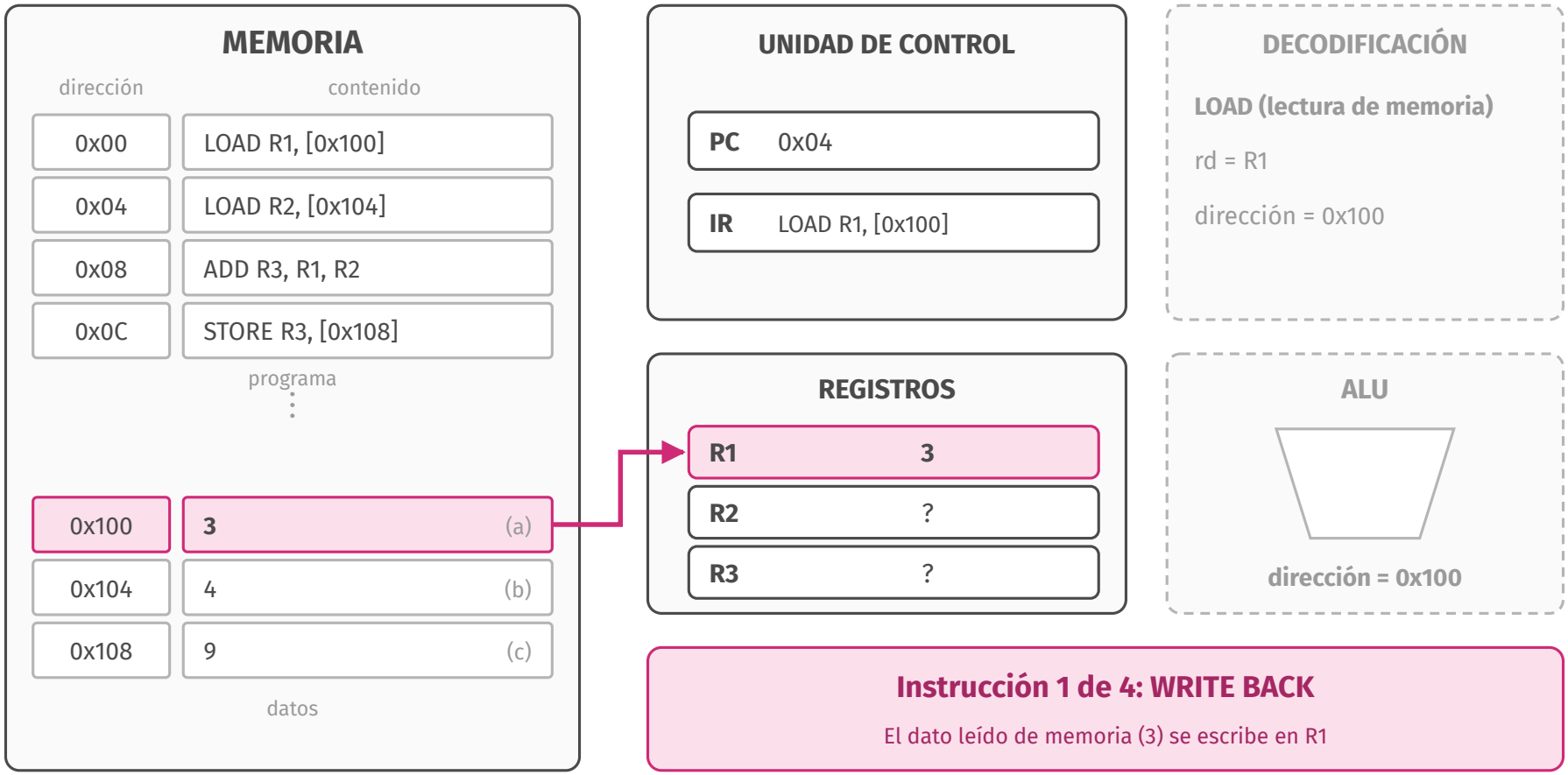


dirección = 0x100

Instrucción 1 de 4: MEMORY

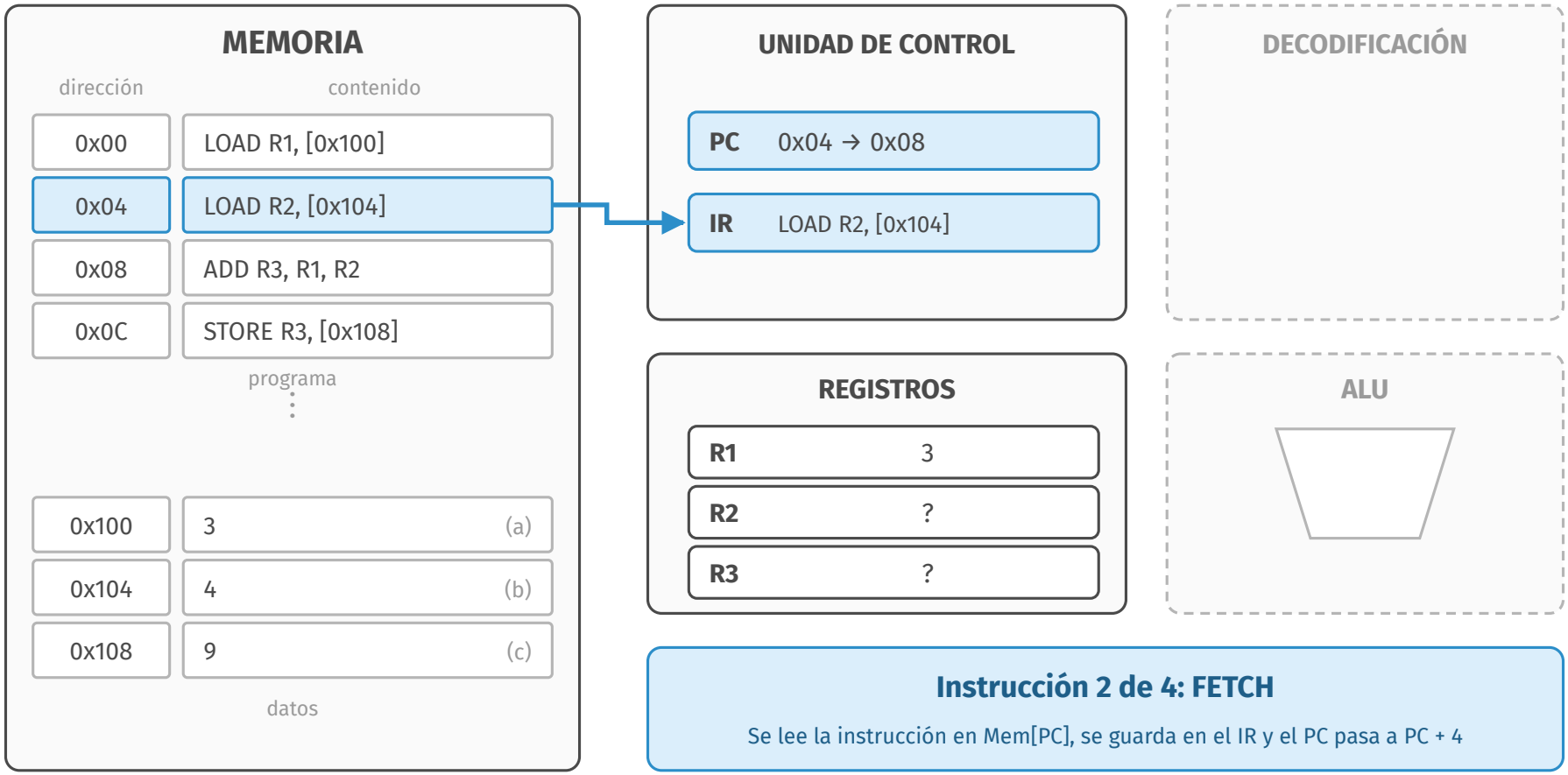
Se lee Mem[0x100]: el dato es 3

Ciclo de instrucción: cómo ejecuta un programa



Ciclo de instrucción: cómo ejecuta un programa

DEPARTAMENTO DE COMPUTACION
Facultad de Ciencias Exactas y Naturales - UBA



Ciclo de instrucción: cómo ejecuta un programa

DEPARTAMENTO DE COMPUTACION
Facultad de Ciencias Exactas y Naturales - UBA

MEMORIA

dirección	contenido	
0x00	LOAD R1, [0x100]	
0x04	LOAD R2, [0x104]	
0x08	ADD R3, R1, R2	
0x0C	STORE R3, [0x108]	

programa

⋮

0x100	3	(a)
0x104	4	(b)
0x108	9	(c)

datos

UNIDAD DE CONTROL

PC	0x08
IR	LOAD R2, [0x104]

REGISTROS

R1	3
R2	?
R3	?

DECODIFICACIÓN

LOAD (lectura de memoria)

rd = R2

dirección = 0x104

ALU

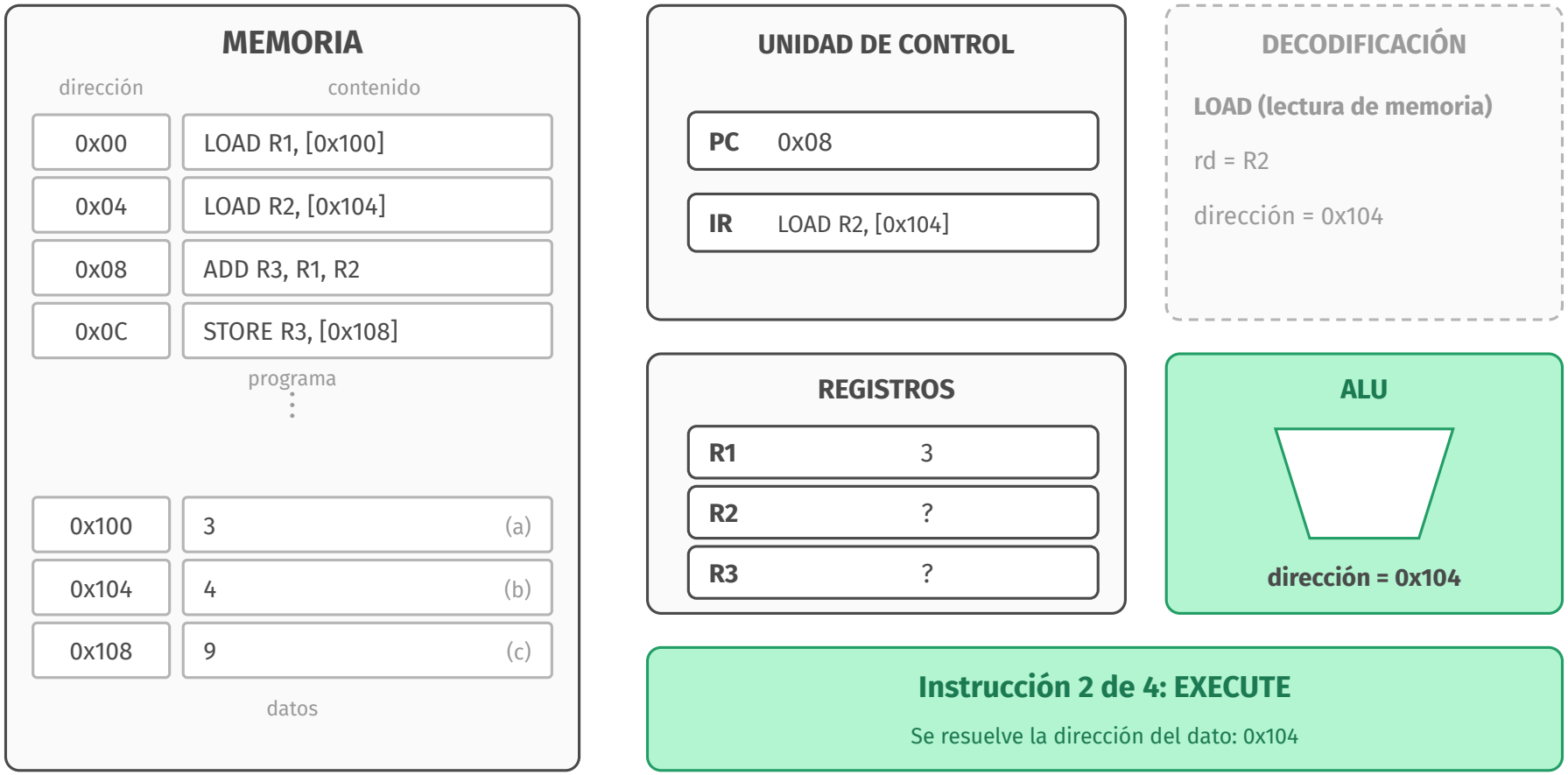


Instrucción 2 de 4: DECODE

Es un load: la dirección del dato viene en la misma instrucción

Ciclo de instrucción: cómo ejecuta un programa

DEPARTAMENTO DE COMPUTACION
Facultad de Ciencias Exactas y Naturales - UBA



Ciclo de instrucción: cómo ejecuta un programa

DEPARTAMENTO DE COMPUTACION
Facultad de Ciencias Exactas y Naturales - UBA

MEMORIA

dirección	contenido
0x00	LOAD R1, [0x100]
0x04	LOAD R2, [0x104]
0x08	ADD R3, R1, R2
0x0C	STORE R3, [0x108]

programa

0x100	3	(a)
0x104	**4**	(b)
0x108	9	(c)

datos

UNIDAD DE CONTROL

PC	0x08
IR	LOAD R2, [0x104]

REGISTROS

R1	3
R2	?
R3	?

DECODIFICACIÓN

LOAD (lectura de memoria)

rd = R2

dirección = 0x104

ALU

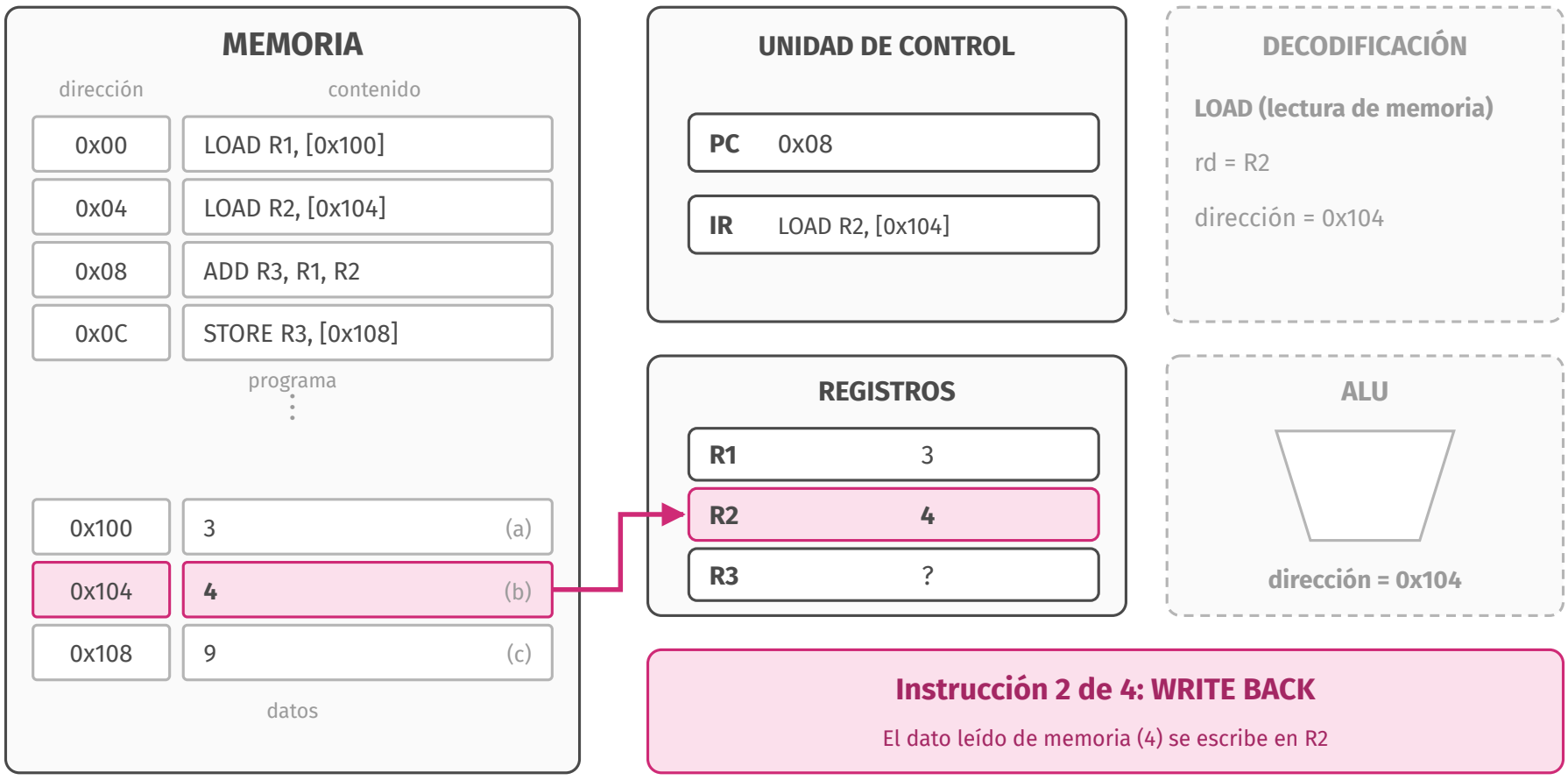


dirección = 0x104

Instrucción 2 de 4: MEMORY

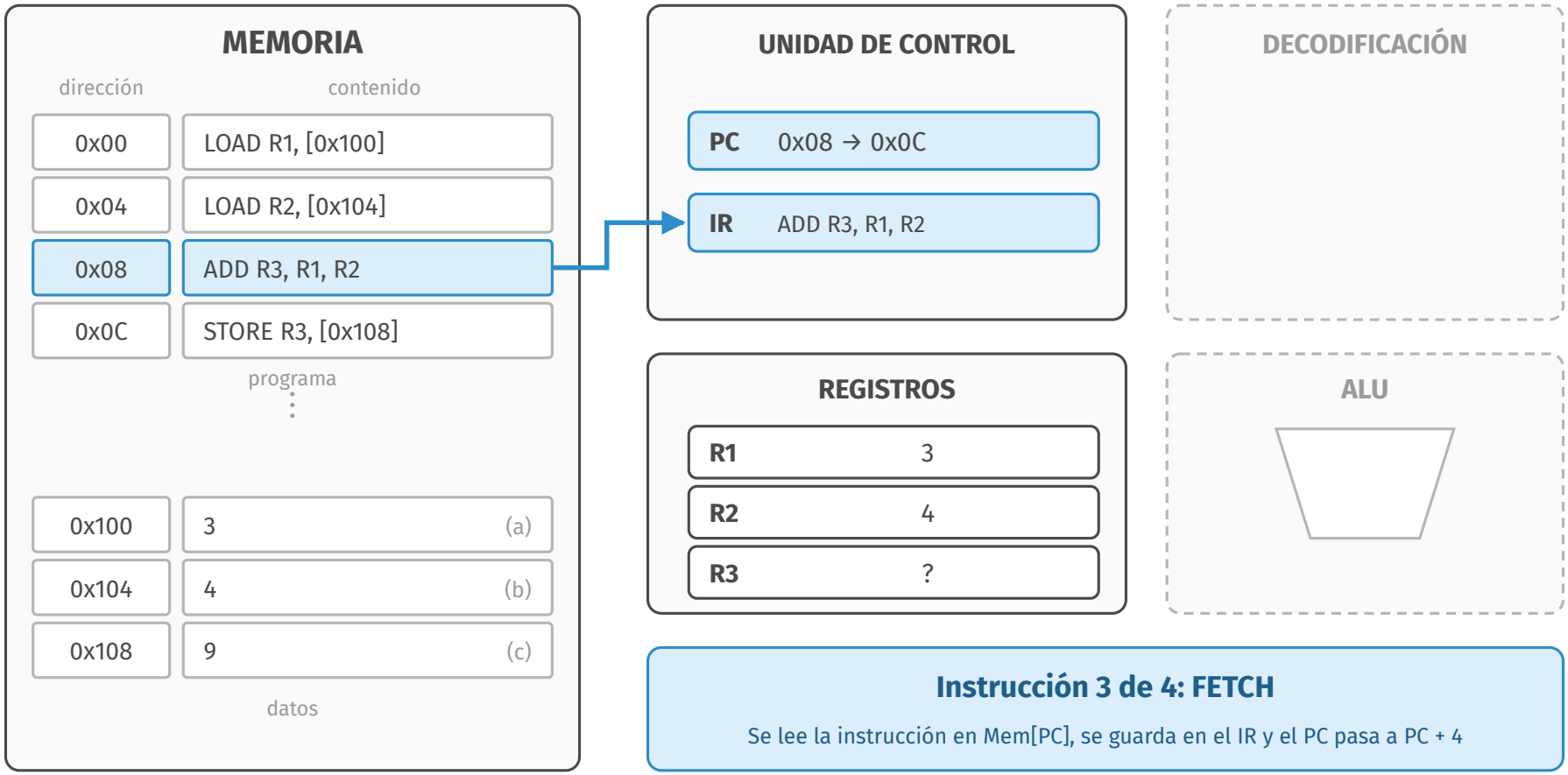
Se lee Mem[0x104]: el dato es 4

Ciclo de instrucción: cómo ejecuta un programa



Ciclo de instrucción: cómo ejecuta un programa

DEPARTAMENTO DE COMPUTACION
Facultad de Ciencias Exactas y Naturales - UBA



Ciclo de instrucción: cómo ejecuta un programa

MEMORIA

dirección	contenido
0x00	LOAD R1, [0x100]
0x04	LOAD R2, [0x104]
0x08	ADD R3, R1, R2
0x0C	STORE R3, [0x108]

programa

⋮

0x100	3	(a)
0x104	4	(b)
0x108	9	(c)

datos

UNIDAD DE CONTROL

PC	0x0C
IR	ADD R3, R1, R2

DECODIFICACIÓN

ADD (suma)

rd = R3

rs1 = R1, rs2 = R2

REGISTROS

R1	**3**
R2	**4**
R3	?

ALU



Instrucción 3 de 4: DECODE

Es una suma: se leen los registros R1 = 3 y R2 = 4

Ciclo de instrucción: cómo ejecuta un programa

DEPARTAMENTO DE COMPUTACION
Facultad de Ciencias Exactas y Naturales - UBA

MEMORIA

dirección	contenido
0x00	LOAD R1, [0x100]
0x04	LOAD R2, [0x104]
0x08	ADD R3, R1, R2
0x0C	STORE R3, [0x108]

programa

⋮

0x100	3	(a)
0x104	4	(b)
0x108	9	(c)

datos

UNIDAD DE CONTROL

PC	0x0C
IR	ADD R3, R1, R2

DECODIFICACIÓN

ADD (suma)

rd = R3

rs1 = R1, rs2 = R2

REGISTROS

R1	3
R2	4
R3	?

ALU



3 + 4 = 7

Instrucción 3 de 4: EXECUTE

La ALU suma: 3 + 4 = 7. ADD no usa la memoria: no pasa por MEMORY

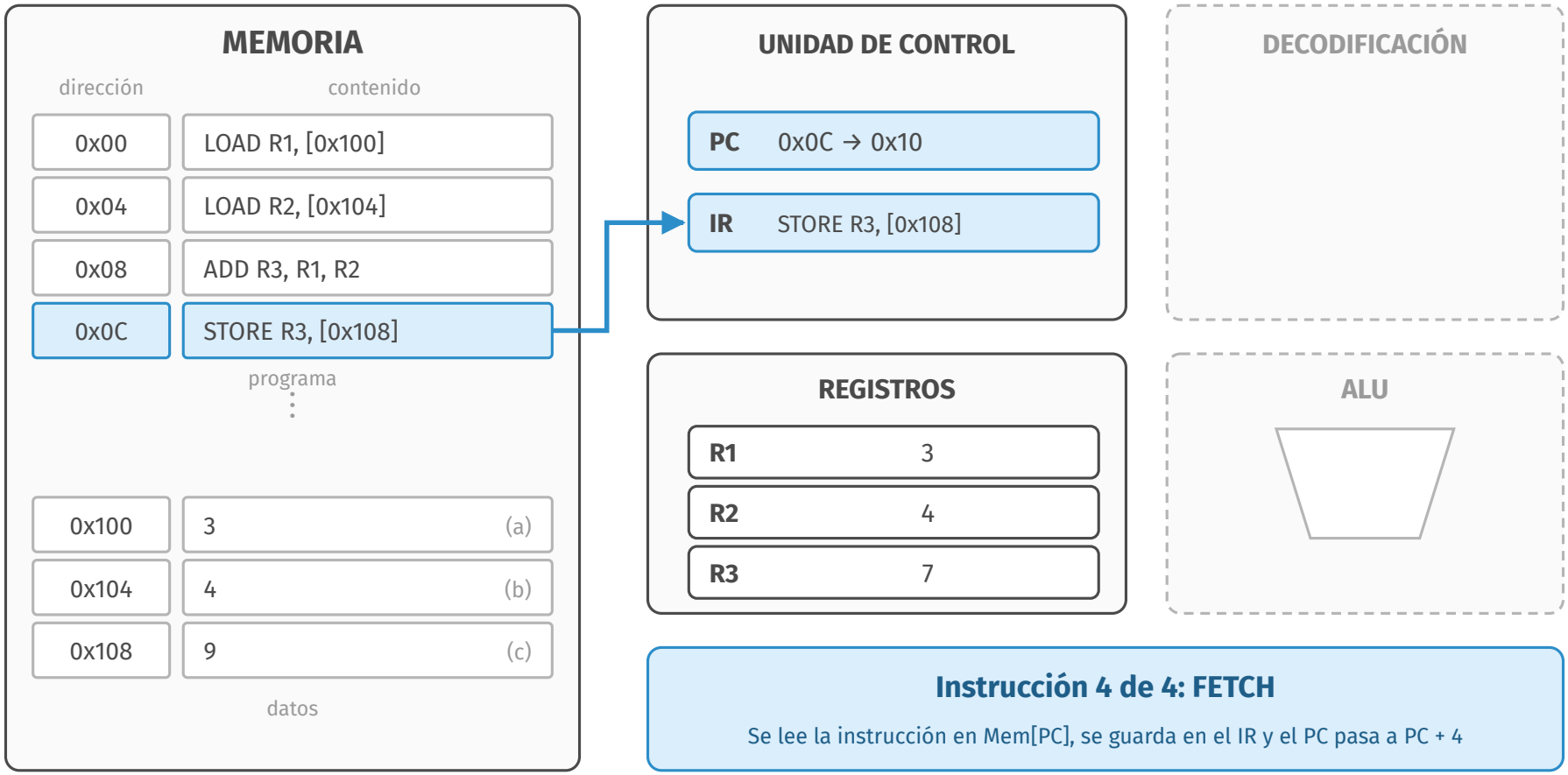
Ciclo de instrucción: cómo ejecuta un programa

DEPARTAMENTO DE COMPUTACION
Facultad de Ciencias Exactas y Naturales - UBA



Ciclo de instrucción: cómo ejecuta un programa

DEPARTAMENTO DE COMPUTACION
Facultad de Ciencias Exactas y Naturales - UBA



Ciclo de instrucción: cómo ejecuta un programa

DEPARTAMENTO DE COMPUTACION
Facultad de Ciencias Exactas y Naturales - UBA

MEMORIA

dirección	contenido	
0x00	LOAD R1, [0x100]	
0x04	LOAD R2, [0x104]	
0x08	ADD R3, R1, R2	
0x0C	STORE R3, [0x108]	

programa

⋮

0x100	3	(a)
0x104	4	(b)
0x108	9	(c)

datos

UNIDAD DE CONTROL

PC	0x10
IR	STORE R3, [0x108]

REGISTROS

R1	3
R2	4
R3	**7**

DECODIFICACIÓN

STORE (escritura)

rs = R3 (el dato)

dirección = 0x108

ALU



Instrucción 4 de 4: DECODE

Es un store: se lee R3 = 7, el dato a guardar

Ciclo de instrucción: cómo ejecuta un programa

DEPARTAMENTO DE COMPUTACION
Facultad de Ciencias Exactas y Naturales - UBA

MEMORIA

dirección	contenido
0x00	LOAD R1, [0x100]
0x04	LOAD R2, [0x104]
0x08	ADD R3, R1, R2
0x0C	STORE R3, [0x108]

programa

0x100	3	(a)
0x104	4	(b)
0x108	9	(c)

datos

UNIDAD DE CONTROL

PC	0x10
IR	STORE R3, [0x108]

DECODIFICACIÓN

STORE (escritura)

rs = R3 (el dato)

dirección = 0x108

REGISTROS

R1	3
R2	4
R3	7

ALU



dirección = 0x108

Instrucción 4 de 4: EXECUTE

Se resuelve la dirección del dato: 0x108

Ciclo de instrucción: cómo ejecuta un programa



El caso RISC-V



Una ISA **abierta**, **modular** y **sin regalías**

RISC-V: características

- Nace en **2010** en la Universidad de Berkeley como un “proyecto de verano” de Krste Asanović, Yunsup Lee y Andrew Waterman, con David Patterson
- La **V**: es la quinta ISA RISC de Berkeley, después de RISC-I (1981), RISC-II, SOAR y SPUR
- **2011**: se publica la primera especificación
- **2015**: se crea la RISC-V Foundation, que en 2020 se muda a Suiza como RISC-V International
- **2019**: se ratifica la ISA base (RV32I / RV64I)
- **Hoy:** abierta y sin regalías, desde microcontroladores hasta servidores



David Patterson
RISC-I, Premio Turing 2017



Andrew Waterman
co-creador de RISC-V, cofundador de SiFive

RISC-V: características

Las decisiones de diseño que vimos para una ISA, y qué eligió RISC-V en cada una:

Decisión	RISC-V eligió	Criterio
Cantidad y complejidad de instrucciones	Pocas y simples: una base de ~40 instrucciones (RV32I)	Costo y simplicidad
Tamaño de instrucción	Fijo, 32 bits (16 bits opcional con la extensión C)	Simplicidad; tamaño del programa
Operandos y registros	3 operandos, 32 registros	Facilidad para programar y compilar
Memoria	Direccionable por byte, *little-endian*	Compatibilidad con el software existente
Modos de direccionamiento	Uno solo: registro base + desplazamiento	Costo y simplicidad
Cómo crece la ISA	Base congelada + extensiones, con opcodes reservados	Espacio para crecer; retrocompatibilidad

RISC-V: características

Qué datos maneja nativamente:

- Los registros tienen **XLEN** bits: 32 en RV32I, 64 en RV64I

RISC-V: características

Qué datos maneja nativamente:

- Los registros tienen **XLEN** bits: 32 en RV32I, 64 en RV64I
- Loads/stores de byte, halfword y word (y doubleword en RV64)

RISC-V: características

Qué datos maneja nativamente:

- Los registros tienen **XLEN** bits: 32 en RV32I, 64 en RV64I
- Loads/stores de byte, halfword y word (y doubleword en RV64)
- Enteros **con y sin signo**: `lb` completa con el signo y `lbu` con ceros; `blt` compara con signo y `bltu` sin signo

RISC-V: instrucciones

Tipo	31...25	24...20	19...15	14...12	11...7	6...0
R	funct7	rs2	rs1	funct3	rd	opcode
I	imm[11:0]		rs1	funct3	rd	opcode
S	imm[11:5]	rs2	rs1	funct3	imm[4:0]	opcode
B	imm[12|10:5]	rs2	rs1	funct3	imm[4:1|11]	opcode
U	imm[31:12]				rd	opcode
J	imm[20|10:1|11|19:12]				rd	opcode

R: registro-registro · **I**: inmediatos cortos y loads · **S**: stores · **B**: branches · **U**: inmediatos largos · **J**: saltos

RISC-V: instrucciones

Tipo	31...25	24...20	19...15	14...12	11...7	6...0
R	funct7	rs2	rs1	funct3	rd	opcode
I	imm[11:0]		rs1	funct3	rd	opcode
S	imm[11:5]	rs2	rs1	funct3	imm[4:0]	opcode
B	imm[12|10:5]	rs2	rs1	funct3	imm[4:1|11]	opcode
U	imm[31:12]				rd	opcode
J	imm[20|10:1|11|19:12]				rd	opcode

R: registro-registro · **I**: inmediatos cortos y loads · **S**: stores · **B**: branches · **U**: inmediatos largos · **J**: saltos

- Solo **6 formatos**, todos de **32 bits** → decodificación simple. x86 tiene muchos formatos: decodificar es más caro

RISC-V: instrucciones

Tipo	31...25	24...20	19...15	14...12	11...7	6...0
R	funct7	rs2	rs1	funct3	rd	opcode
I	imm[11:0]		rs1	funct3	rd	opcode
S	imm[11:5]	rs2	rs1	funct3	imm[4:0]	opcode
B	imm[12|10:5]	rs2	rs1	funct3	imm[4:1|11]	opcode
U	imm[31:12]				rd	opcode
J	imm[20|10:1|11|19:12]				rd	opcode

R: registro-registro · **I**: inmediatos cortos y loads · **S**: stores · **B**: branches · **U**: inmediatos largos · **J**: saltos

- Solo **6 formatos**, todos de **32 bits** → decodificación simple. x86 tiene muchos formatos: decodificar es más caro
- **3 registros** por instrucción (dos fuentes + un destino): no hay que compartir un campo entre fuente y destino, como en x86-32

RISC-V: instrucciones

Tipo	31...25	24...20	19...15	14...12	11...7	6...0
R	funct7	rs2	rs1	funct3	rd	opcode
I	imm[11:0]		rs1	funct3	rd	opcode
S	imm[11:5]	rs2	rs1	funct3	imm[4:0]	opcode
B	imm[12|10:5]	rs2	rs1	funct3	imm[4:1|11]	opcode
U	imm[31:12]				rd	opcode
J	imm[20|10:1|11|19:12]				rd	opcode

R: registro-registro · **I**: inmediatos cortos y loads · **S**: stores · **B**: branches · **U**: inmediatos largos · **J**: saltos

- Solo **6 formatos**, todos de **32 bits** → decodificación simple. x86 tiene muchos formatos: decodificar es más caro
- **3 registros** por instrucción (dos fuentes + un destino): no hay que compartir un campo entre fuente y destino, como en x86-32
- Los campos de registro (■) están en la **misma posición** en todos los formatos → se pueden empezar a leer los registros antes de terminar de decodificar

RISC-V: instrucciones

Tipo	Instrucciones (RV32I)	Formato
Aritméticas	`add`, `sub`, `addi`	R / I
Lógicas	`and`, `or`, `xor`, `andi`, `ori`, `xori`	R / I
Desplazamientos	`sll`, `srl`, `sra`, `slli`, `srli`, `srai`	R / I
Comparación	`slt`, `sltu`, `slti`, `sltiu`	R / I
Inmediatos largos	`lui`, `auipc`	U
Lectura de memoria	`lb`, `lh`, `lw`, `lbu`, `lhu`	I
Escritura en memoria	`sb`, `sh`, `sw`	S
Saltos condicionales	`beq`, `bne`, `blt`, `bge`, `bltu`, `bgeu`	B
Saltos incondicionales	`jal`, `jalr`	J / I
Sistema	`ecall`, `ebreak`, `fence`	I

Patterson & Waterman, The RISC-V Reader, *2017, cap. 2*

RISC-V: instrucciones


```
add s1, s2, s3
```

s1 = s2 + s3

RISC-V: instrucciones


```
add s1, s2, s3
```

s1 = s2 + s3

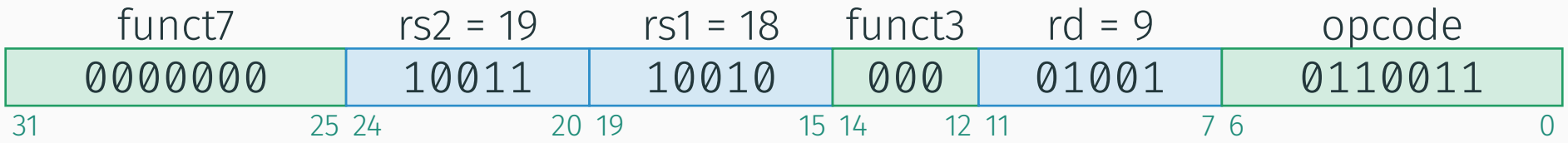
- Formato **R**. Cada registro se escribe con su número: `s1` = `x9`, `s2` = `x18`, `s3` = `x19`
- El opcode y los `funct` dicen que es un `add`

RISC-V: instrucciones


```
add s1, s2, s3
```

s1 = s2 + s3

- Formato **R**. Cada registro se escribe con su número: `s1` = `x9`, `s2` = `x18`, `s3` = `x19`
- El opcode y los `funct` dicen que es un `add`

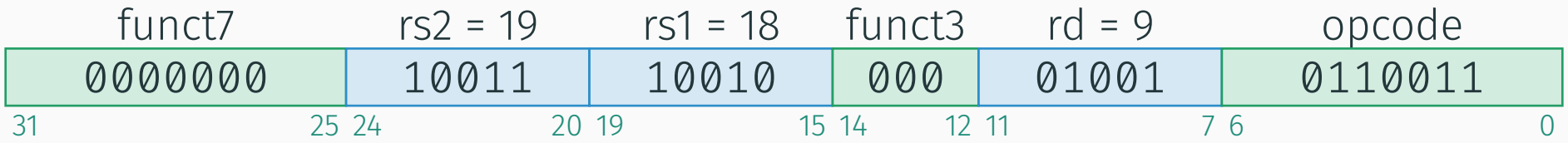


RISC-V: instrucciones


```
add s1, s2, s3
```

s1 = s2 + s3

- Formato **R**. Cada registro se escribe con su número: `s1` = `x9`, `s2` = `x18`, `s3` = `x19`
- El opcode y los `funct` dicen que es un `add`



En memoria: `0×013904B3`

RISC-V: instrucciones


```
lw s3, 8(s0)
```

s3 = Mem[s0 + 8]

RISC-V: instrucciones


```
lw s3, 8(s0)
```

s3 = Mem[s0 + 8]

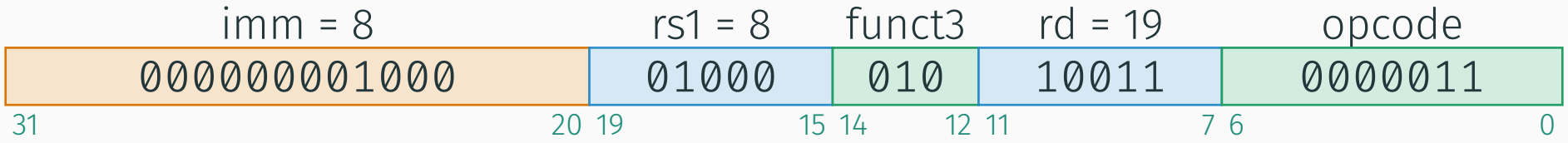
- Formato I. `s3` = `x19`, `s0` = `x8`, y el desplazamiento 8 va en el inmediato

RISC-V: instrucciones


```
lw s3, 8(s0)
```

s3 = Mem[s0 + 8]

- Formato **I**. `s3` = `x19`, `s0` = `x8`, y el desplazamiento 8 va en el inmediato

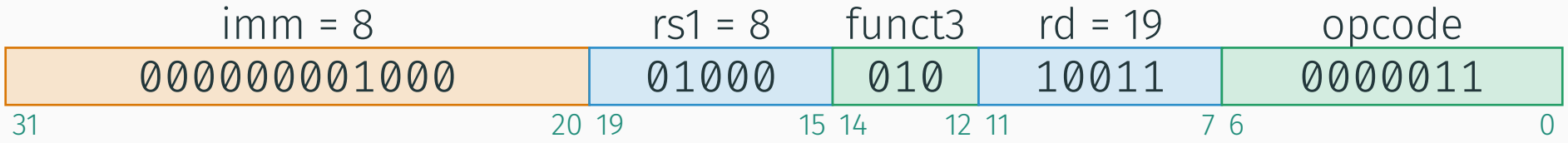


RISC-V: instrucciones


```
lw s3, 8(s0)
```

s3 = Mem[s0 + 8]

- Formato I. `s3` = `x19`, `s0` = `x8`, y el desplazamiento 8 va en el inmediato



En memoria: `0×00842983`

RISC-V: instrucciones

Una instrucción ocupa 32 bits: no queda lugar para un inmediato de 32 bits. Por eso una constante grande se carga en dos partes

RISC-V: instrucciones

Una instrucción ocupa 32 bits: no queda lugar para un inmediato de 32 bits. Por eso una constante grande se carga en dos partes

Por ejemplo, para cargar `0×CAFEF00D` en `t0`, primero `lui` y después `addi`:

RISC-V: instrucciones

Una instrucción ocupa 32 bits: no queda lugar para un inmediato de 32 bits. Por eso una constante grande se carga en dos partes

Por ejemplo, para cargar `0×CAFEF00D` en `t0`, primero `lui` y después `addi`:

```
lui  t0, 0×CAFEF
```



RISC-V: instrucciones

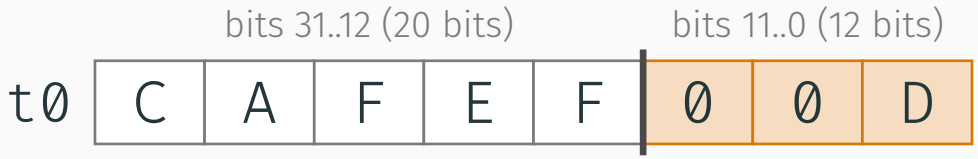
Una instrucción ocupa 32 bits: no queda lugar para un inmediato de 32 bits. Por eso una constante grande se carga en dos partes

Por ejemplo, para cargar `0×CAFEF00D` en `t0`, primero `lui` y después `addi`:

```
lui  t0, 0×CAFEF
```




```
addi t0, t0, 0×00D
```



RISC-V: instrucciones

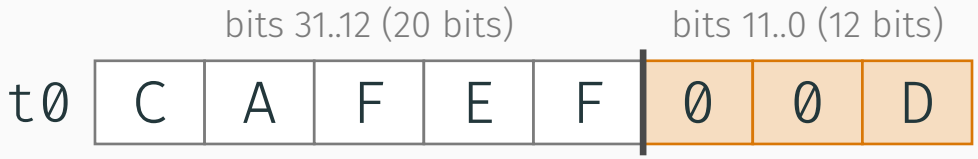
Una instrucción ocupa 32 bits: no queda lugar para un inmediato de 32 bits. Por eso una constante grande se carga en dos partes

Por ejemplo, para cargar `0×CAFEF00D` en `t0`, primero `lui` y después `addi`:

```
lui  t0, 0×CAFEF
```




```
addi t0, t0, 0×00D
```



- `lui` (formato **U**) pone 20 bits en la parte alta (`0×CAFEF << 12`) y ceros abajo; `addi` completa los 12 de abajo
- `auipc` es igual, pero le suma el PC: sirve para direcciones lejanas

RISC-V: instrucciones

¿Y para cargar `0×DEADBEEF`? El inmediato de `addi` tiene signo: `0×EEF` tiene el bit más alto en 1, así que vale `-0×111`

RISC-V: instrucciones

¿Y para cargar `0×DEADBEEF`? El inmediato de `addi` tiene signo: `0×EEF` tiene el bit más alto en 1, así que vale `-0×111`

Entonces se carga una parte alta de más y después se resta:

```
lui  t0, 0×DEADC
```



RISC-V: instrucciones

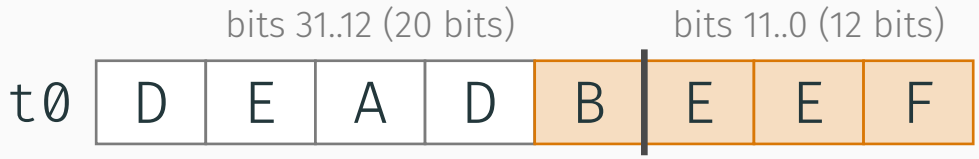
¿Y para cargar `0×DEADBEEF`? El inmediato de `addi` tiene signo: `0×EEF` tiene el bit más alto en 1, así que vale `-0×111`

Entonces se carga una parte alta de más y después se resta:

```
lui  t0, 0×DEADC
```




```
addi t0, t0, -0×111
```



RISC-V: instrucciones

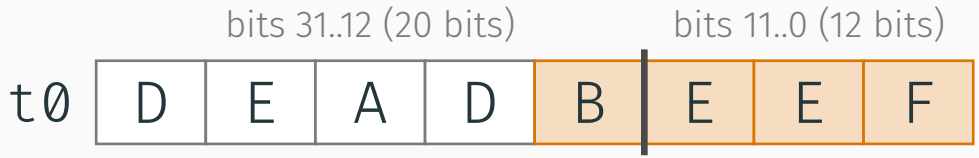
¿Y para cargar `0×DEADBEEF`? El inmediato de `addi` tiene signo: `0×EEF` tiene el bit más alto en 1, así que vale `-0×111`

Entonces se carga una parte alta de más y después se resta:

```
lui  t0, 0×DEADC
```




```
addi t0, t0, -0×111
```



`0×DEADC000 - 0×111 = 0×DEADBEEF`: la resta también cambia el último dígito de la parte alta (C → B)

RISC-V: registros

32 registros (`x0`...`x31`) de XLEN bits, todos de **propósito general**, salvo `x0`: está fijo en 0 (siempre se lee 0 y lo que se escribe se descarta)

x0 = 0	x1	x2	x3	x4	x5	x6	x7
x8	x9	x10	x11	x12	x13	x14	x15
x16	x17	x18	x19	x20	x21	x22	x23
x24	x25	x26	x27	x28	x29	x30	x31

RISC-V: registros

En assembly se usan los nombres de la **ABI**: una convención de software para que los programas se pongan de acuerdo en qué registro usar para qué. No la define la ISA

Registro(s)	Nombre ABI	Uso
x0	zero	Conectado a 0: siempre se lee 0 y las escrituras se descartan
x1	ra	Dirección de retorno
x2	sp	Stack pointer
x3	gp	Global pointer
x4	tp	Thread pointer
x5–x7, x28–x31	t0–t6	Temporales
x8–x9, x18–x27	s0–s11	Saved registers (s0 = frame pointer)
x10–x17	a0–a7	Argumentos de función / valor de retorno

RISC-V: control de flujo

No importa si el salto es condicional o no: lo que cambia es **cómo se calcula el destino**

Salto	Destino	Instrucciones	Para qué
Relativo al PC	`PC ← PC + imm`	`beq`, `bne`, `blt`, `bge`... `jal`	`if`, ciclos, llamar a una función: el destino se conoce al ensamblar
Por registro	`PC ← R[rs1] + imm`	`jalr`	volver de una función: el destino se conoce recién al ejecutar

RISC-V: control de flujo

No importa si el salto es condicional o no: lo que cambia es **cómo se calcula el destino**

Salto	Destino	Instrucciones	Para qué
Relativo al PC	`PC ← PC + imm`	`beq`, `bne`, `blt`, `bge`... `jal`	`if`, ciclos, llamar a una función: el destino se conoce al ensamblar
Por registro	`PC ← R[rs1] + imm`	`jalr`	volver de una función: el destino se conoce recién al ejecutar

- En un salto relativo, el ensamblador traduce la etiqueta a la **distancia** desde la instrucción: `imm = etiqueta - PC`. El programa funciona en cualquier dirección donde se cargue
- `jal` y `jalr` además guardan en `rd` la dirección siguiente (`PC + 4`): adónde volver. Con `rd = zero` no se guarda nada: es un salto simple

RISC-V: control de flujo


```
0×00  main: addi a0, zero, 3
0×04        jal  ra, doble
0×08        addi t0, zero, 6
0×0C        beq  a0, t0, fin
0×10        addi a0, zero, 0
0×14   fin: ...
0×18 doble: add  a0, a0, a0
0×1C        jalr zero, 0(ra)
```

Registro	Valor
PC	0×00
ra	?
a0	?
t0	?

El programa está en memoria. Las etiquetas son alias de direcciones: `main` = `0×00`, `fin` = `0×14`, `doble` = `0×18`

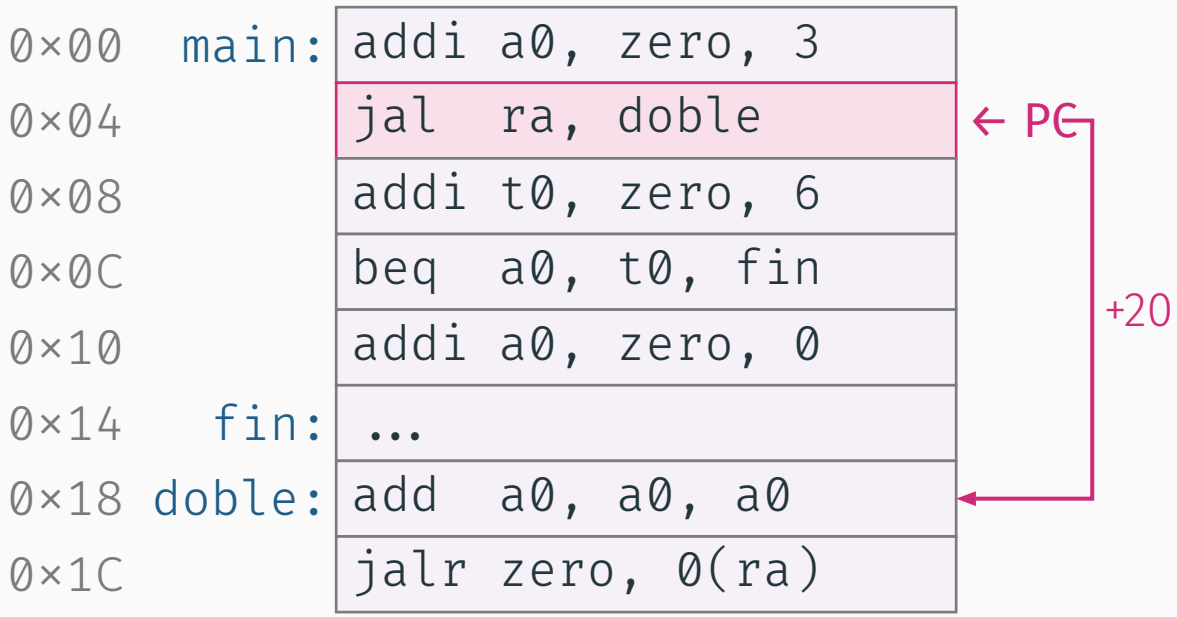
RISC-V: control de flujo


```
0×00  main: addi a0, zero, 3
0×04        jal  ra, doble      ← PC
0×08        addi t0, zero, 6
0×0C        beq  a0, t0, fin
0×10        addi a0, zero, 0
0×14   fin: ...
0×18 doble: add  a0, a0, a0
0×1C        jalr zero, 0(ra)
```

Registro	Valor
PC	0×04
ra	?
a0	3
t0	?

`addi a0, zero, 3` (`a0 = 0 + 3`): no es un salto, así que el PC avanza a la siguiente: `PC + 4`

RISC-V: control de flujo



Registro	Valor
PC	0×18
ra	0×08
a0	3
t0	?

`jal ra, doble` **relativo**: el ensamblador guardó `imm = 0×18 - 0×04 = 20`. Guarda `ra ← 0×08` y salta a `PC + 20 = 0×18`

RISC-V: control de flujo

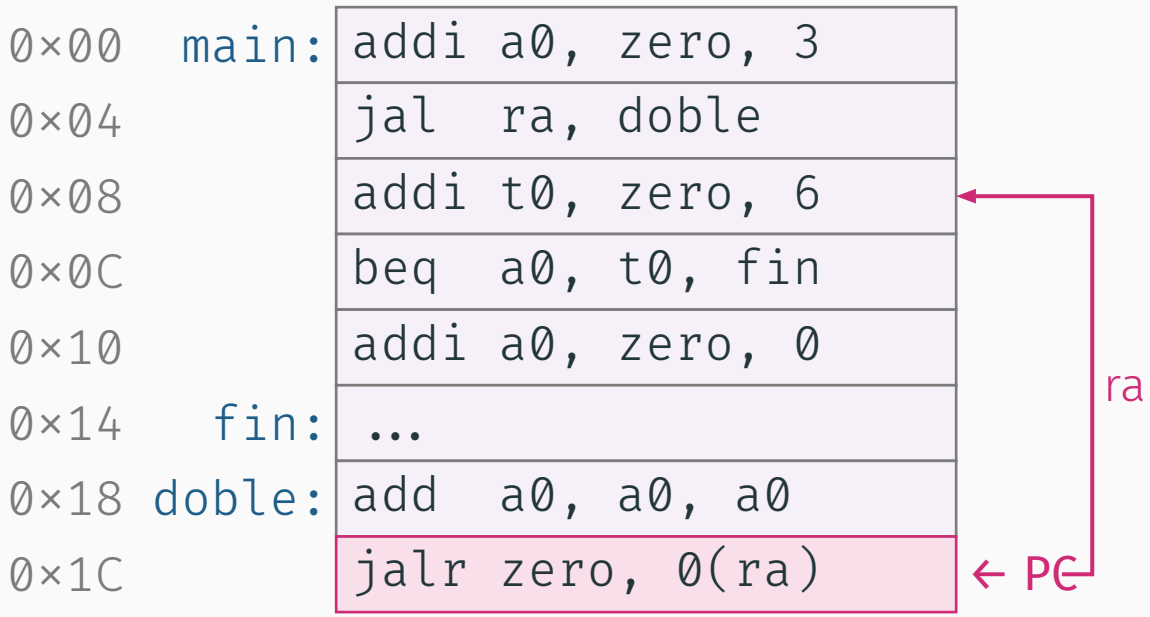
DEPARTAMENTO DE COMPUTACION
Facultad de Ciencias Exactas y Naturales - UBA

```
0×00   main:  addi a0, zero, 3
0×04          jal  ra, doble
0×08          addi t0, zero, 6
0×0C          beq  a0, t0, fin
0×10          addi a0, zero, 0
0×14    fin:  ...
0×18  doble:  add  a0, a0, a0
0×1C          jalr zero, 0(ra)   ← PC
```

Registro	Valor
PC	0×1C
ra	0×08
a0	6
t0	?

`add a0, a0, a0`: `a0 = 6`. Sigue en secuencia

RISC-V: control de flujo



Registro	Valor
PC	0×08
ra	0×08
a0	6
t0	?

`jalr zero, 0(ra)` **por registro**: salta a `ra + 0 = 0×08`. La instrucción no sabe adónde vuelve: si `doble` se llamara desde otro lado, `ra` tendría otra dirección

RISC-V: control de flujo

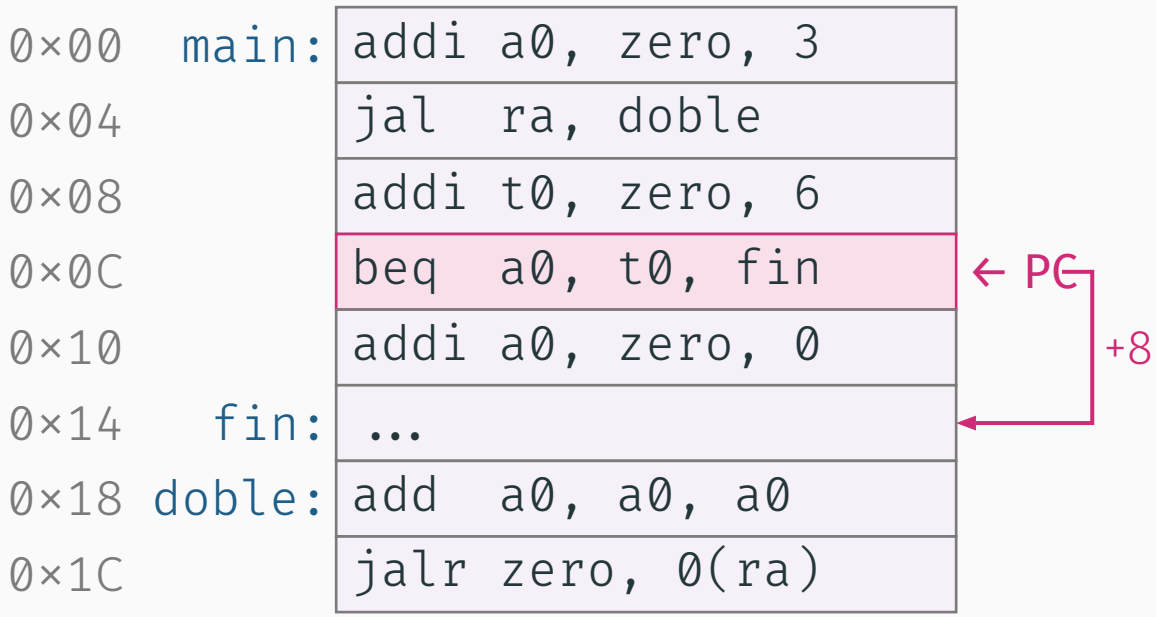
DEPARTAMENTO DE COMPUTACION
Facultad de Ciencias Exactas y Naturales - UBA

```
0×00  main: addi a0, zero, 3
0×04        jal  ra, doble
0×08        addi t0, zero, 6
0×0C        beq  a0, t0, fin   ← PC
0×10        addi a0, zero, 0
0×14   fin: ...
0×18 doble: add  a0, a0, a0
0×1C        jalr zero, 0(ra)
```

Registro	Valor
PC	0×0C
ra	0×08
a0	6
t0	6

`addi t0, zero, 6` y sigue en secuencia hasta el `beq`

RISC-V: control de flujo



Registro	Valor
PC	0×14
ra	0×08
a0	6
t0	6

`beq a0, t0, fin` **relativo**: `a0 = t0`, entonces salta a `PC + 8 = 0×14` (`imm = 0×14 - 0×0C`). Si no fueran iguales, seguiría en `0×10`

RISC-V: extensiones

> ***Extensión:*** *un conjunto de capacidades nuevas que se suma a la ISA base: instrucciones, registros, etc.*

Cada una se nombra con una letra o con `Z` **...** :

Categoría	Extensiones	Qué agregan
Aritmética entera	M, B	Multiplicación y división; manipulación de bits
Punto flotante	F, D, Q, Zfh	Precisión simple, doble, cuádruple y media
Paralelismo	V, A	Vectores (SIMD); operaciones atómicas para multiprocesadores
Tamaño de código	C	Instrucciones comprimidas de 16 bits
Sistema	Zicsr, Zifencei, H	Registros de control y estado; sincronización; hipervisor
Seguridad	Zk...	Criptografía (AES, SHA)

RISC-V: extensiones

Con tantas extensiones opcionales, ¿qué puede asumir un programa ya compilado (por ejemplo, una distribución de Linux)?

RISC-V: extensiones

Con tantas extensiones opcionales, ¿qué puede asumir un programa ya compilado (por ejemplo, una distribución de Linux)?

> ***Perfil:*** *un conjunto de extensiones con nombre: para cumplirlo, un procesador tiene que implementarlas todas.*

RISC-V: extensiones

Con tantas extensiones opcionales, ¿qué puede asumir un programa ya compilado (por ejemplo, una distribución de Linux)?

> ***Perfil:*** *un conjunto de extensiones con nombre: para cumplirlo, un procesador tiene que implementarlas todas.*

RVA20	`RV64I` `M` `A` `F` `D` `C`	2020: lo mínimo para Linux (RV64GC)
RVA22	`RV64I` `MAFDC` `Zba` `Zbb` `Zbs` `...`	2022: suma manipulación de bits
RVA23	`RV64I` `MAFDC` `Zb...` `V` `H`	2023: suma vectores e hipervisor

RISC-V: extensiones

Con tantas extensiones opcionales, ¿qué puede asumir un programa ya compilado (por ejemplo, una distribución de Linux)?

> ***Perfil:*** *un conjunto de extensiones con nombre: para cumplirlo, un procesador tiene que implementarlas todas.*

RVA20	`RV64I` `M` `A` `F` `D` `C`	2020: lo mínimo para Linux (RV64GC)
RVA22	`RV64I` `MAFDC` `Zba Zbb Zbs` `...`	2022: suma manipulación de bits
RVA23	`RV64I` `MAFDC` `Zb ...` `V` `H`	2023: suma vectores e hipervisor

- Cada perfil incluye al anterior y suma extensiones (□): el software compilado para RVA20 corre en un procesador RVA23
- Un microcontrolador no necesita ningún perfil de aplicación: elige solo las extensiones que le sirven

RISC-V: el mismo programa

Genérico

```
suma:                   # R1 = a, R2 = n
    LI    R3, 0         # acum = 0
    LI    R4, 0         # i = 0
ciclo:
    BGE   R4, R2, fin   # si i ⩾ n, salir
    SLLI  R5, R4, 2     # R5 = i * 4
    ADD   R5, R1, R5    # R5 = &a[i]
    LOAD  R5, 0(R5)     # R5 = a[i]
    ADD   R3, R3, R5    # acum += a[i]
    ADDI  R4, R4, 1     # i++
    JUMP  ciclo         # volver a la condición
fin:
    MOV   R1, R3        # valor de retorno
    RET
```

RISC-V

```
suma:                   # a0 = a, a1 = n
    addi t0, zero, 0    # acum = 0
    addi t1, zero, 0    # i = 0
ciclo:
    bge  t1, a1, fin    # si i ⩾ n, salir
    slli t2, t1, 2      # t2 = i * 4
    add  t2, a0, t2     # t2 = &a[i]
    lw   t2, 0(t2)      # t2 = a[i]
    add  t0, t0, t2     # acum += a[i]
    addi t1, t1, 1      # i++
    jal  zero, ciclo    # volver a la condición
fin:
    addi a0, t0, 0      # valor de retorno
    jalr zero, 0(ra)
```

RISC-V: el mismo programa

Genérico

```
suma:                   # R1 = a, R2 = n
    LI    R3, 0         # acum = 0
    LI    R4, 0         # i = 0
ciclo:
    BGE   R4, R2, fin   # si i ⩾ n, salir
    SLLI  R5, R4, 2     # R5 = i * 4
    ADD   R5, R1, R5    # R5 = &a[i]
    LOAD  R5, 0(R5)     # R5 = a[i]
    ADD   R3, R3, R5    # acum += a[i]
    ADDI  R4, R4, 1     # i++
    JUMP  ciclo         # volver a la condición
fin:
    MOV   R1, R3        # valor de retorno
    RET
```

RISC-V

```
suma:                   # a0 = a, a1 = n
    addi t0, zero, 0    # acum = 0
    addi t1, zero, 0    # i = 0
ciclo:
    bge  t1, a1, fin    # si i ⩾ n, salir
    slli t2, t1, 2      # t2 = i * 4
    add  t2, a0, t2     # t2 = &a[i]
    lw   t2, 0(t2)      # t2 = a[i]
    add  t0, t0, t2     # acum += a[i]
    addi t1, t1, 1      # i++
    jal  zero, ciclo    # volver a la condición
fin:
    addi a0, t0, 0      # valor de retorno
    jalr zero, 0(ra)
```

- Registros con su nombre de la ABI: argumentos y retorno en `a0`, `a1`; temporales en `t0`…`t2`
- `LOAD` de una palabra es `lw`. RISC-V no tiene `LI`, `MOV`, `JUMP` ni `RET`: se hacen con `addi`, `jal` y `jalr`, usando `zero`

Cierre

Conclusión

- **Von Neumann**: programa y datos en una misma memoria; el PC recorre las instrucciones de a una

Conclusión

- **Von Neumann**: programa y datos en una misma memoria; el PC recorre las instrucciones de a una
- **ISA**: el contrato entre hardware y software
  - Define instrucciones, registros, palabra, modos de direccionamiento y endianness
  - Cada instrucción se codifica en bits según un formato
  - Las decisiones de un programa se implementan con saltos

Conclusión

- **Von Neumann**: programa y datos en una misma memoria; el PC recorre las instrucciones de a una
- **ISA**: el contrato entre hardware y software
  - Define instrucciones, registros, palabra, modos de direccionamiento y endianness
  - Cada instrucción se codifica en bits según un formato
  - Las decisiones de un programa se implementan con saltos
- **Ciclo de instrucción**: fetch → decode → execute → memory → write back, una y otra vez

Conclusión

- **Von Neumann**: programa y datos en una misma memoria; el PC recorre las instrucciones de a una
- **ISA**: el contrato entre hardware y software
  - Define instrucciones, registros, palabra, modos de direccionamiento y endianness
  - Cada instrucción se codifica en bits según un formato
  - Las decisiones de un programa se implementan con saltos
- **Ciclo de instrucción**: fetch → decode → execute → memory → write back, una y otra vez
- **RISC-V**: una ISA abierta y modular, que es la que vamos a usar

Bibliografía

Para profundizar en los temas de hoy:

- J. L. Hennessy y D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6ta ed., Morgan Kaufmann, 2017.
  - Apéndice A: principios de diseño de una ISA
- S. L. Harris y D. M. Harris, *Digital Design and Computer Architecture: RISC-V Edition*, Morgan Kaufmann, 2021.
  - Capítulo 6: arquitectura (ISA de RISC-V)
- Y. N. Patt y S. J. Patel, *Introduction to Computing Systems: From Bits & Gates to C/C++ & Beyond*, 3ra ed., McGraw-Hill, 2020.
  - Capítulo 4: el modelo de Von Neumann
- D. Patterson y A. Waterman, *The RISC-V Reader: An Open Architecture Atlas*, Strawberry Canyon, 2017.

¿Preguntas?